42 NETWORK · COMMON CORE · RANK 00

Libft

The Companion — every function, struct and type, explained for the defense and for recoding.

Repository studied: **github.com/hamzahdili2001/42-Libft-Version-19.3**
43 functions · 1 struct · 1 Makefile · traced, tested and audited
Prepared for **Laia Tabbiz** · 1337 · October 2026



A STRING IS JUST BYTES UNTIL YOU MEET THE \0

Contents

1 · The Project at a Glance 3

2 · C Foundations for Libft 6

1. Where data lives 6
2. The data types used in this library 7
3. Pointers, arrays and strings 8
4. malloc and free — the contract 9
5. Function pointers 9
6. static helpers, headers and include guards 10
7. File descriptors and write 10
8. Integer limits and INT_MIN 10

3 · libft.h, Line by Line 12

The include guard (lines 1–2 and the last line) 13
The system includes 13
The struct: typedef struct s_list 13
The prototypes 14

4 · Character Functions 16

`ft_isalpha` 16
`ft_isdigit` 17
`ft_isalnum` 17
`ft_isascii` 18
`ft_isprint` 18
`ft_toupper` 19
`ft_tolower` 19

5 · Memory Functions 21

`ft_memset` 21
`ft_bzero` 22
`ft_memcpy` 23
`ft_memmove` 24
`ft_memchr` 25
`ft_memcmp` 26
`ft_calloc` 27

6 · String Functions 29

`ft_strlen` 29
`ft_strlcpy` 30
`ft_strlcat` 31
`ft_strchr` 32
`ft_strrchr` 33
`ft_strncmp` 34
`ft_strnstr` 35
`ft_strdup` 36
`ft_atoi` 37

7 · Part 2: New Strings & Callbacks 40

`ft_substr` 40
`ft_strjoin` 41
`ft_strtrim` 43
`ft_split` 44
`ft_itoa` 47
`ft_strmapi` 48
`ft_striteri` 49

8 · Output Functions 51

`ft_putchar_fd` 51
`ft_putstr_fd` 52
`ft_putendl_fd` 52
`ft_putnbr_fd` 53

9 · The t_list Struct & Linked Lists 55

Why a linked list? 55
The struct in action 55
t_list * versus t_list ** 56
`ft_lstnew` 57
`ft_lstadd_front` 57
`ft_lstsize` 58
`ft_lstlast` 59
`ft_lstadd_back` 59
`ft_lstdelone` 60
`ft_lstclear` 61
`ft_lstiter` 62
`ft_lstmap` 62

10 · The Makefile 65

How make decides what to do 66
Using the library 67

11 · Audit: What Breaks in This Version 69

Compiler output on the original files 70
The three logic fixes, side by side 70
Verification of the corrected version 70

12 · Defense Preparation & Recoding Drills 72

How the evaluation usually goes 72
General questions 72
What testers check (be ready) 73
Recall cards — rewrite from memory 73

Appendix A · Complete Corrected Source 76

CHAPTER 1

The Project at a Glance

> *What Libft is, how its 43 functions depend on each other, and how a pile of .c files becomes libft.a.*

Libft is the first project after the Piscine: you rewrite a set of standard C library functions, add a few extra string utilities, and implement a singly linked list. All of it is compiled into one **static library**, `libft.a`, which you will link into later projects (ft_printf, get_next_line, push_swap…). This book walks through the specific repository `hamzahdili2001/42-Libft-Version-19.3` file by file. For every function you get: what it is for, what each type in its prototype means, a diagram of what happens in memory, the code exactly as written in the repo, a step-by-step trace, the edge cases, and the questions an evaluator is likely to ask.

The three parts

PART	WHAT IT IS	FUNCTIONS IN THIS REPO
1 · Libc	Re-implementations of standard functions. Same behaviour as the real ones (check `man 3 strlcpy` …), prefixed with `ft_`.	isalpha isdigit isalnum isascii isprint toupper tolower · strlen strlcpy strlcat strchr strrchr strncmp strnstr strdup atoi · memset bzero memcpy memmove memchr memcmp calloc
2 · Additional	Functions that do not exist in libc (or exist differently). Most of them **allocate** a new string with malloc.	substr strjoin strtrim split itoa strmapi striteri putchar_fd putstr_fd putendl_fd putnbr_fd
3 · Lists	A singly linked list built on the `t_list` struct declared in `libft.h`.	lstnew lstadd_front lstadd_back lstsize lstlast lstdelone lstclear lstiter lstmap

IN THIS REPO

This specific version of the repository **does not compile as published**: 22 of the 43 `.c` files are missing `#include "libft.h"`, the Makefile has no rules, `ft_strdup.c` still contains a `main()`, and `ft_strjoin` has no prototype. These are documented in Chapter 11 with exact fixes, and Appendix A gives the full corrected source. Everything in this book was verified by building the corrected version with `cc -Wall -Wextra -Werror` and comparing the output against the real libc.

Who calls whom

Functions in Libft are built on each other. Knowing this graph matters in a defense: if an evaluator asks you to change `ft_strlen`, you should know that 9 other functions are affected. Arrows point from a caller to the function it uses.

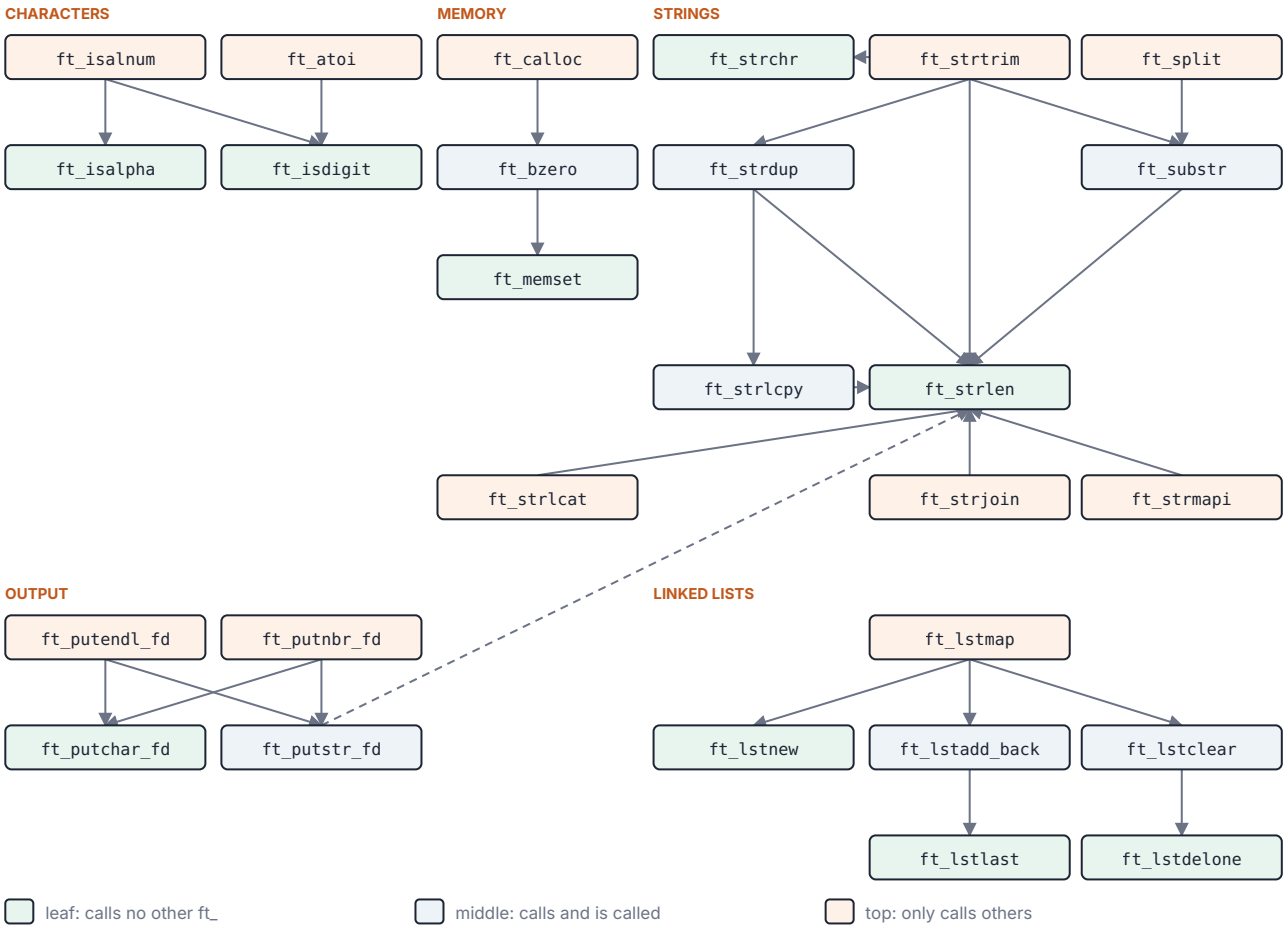


Figure — Internal dependencies of the library (static helpers not shown)

From .c files to a program

Each `.c` file is compiled separately into an **object file** (`.o`) — machine code that still has unresolved names. `ar rcs` bundles all objects into one archive. When someone links against `libft.a`, the linker copies *only the object files that are actually needed* into the final executable. That is the meaning of "static library": the code is copied in at link time, not loaded at run time like a `.so`.

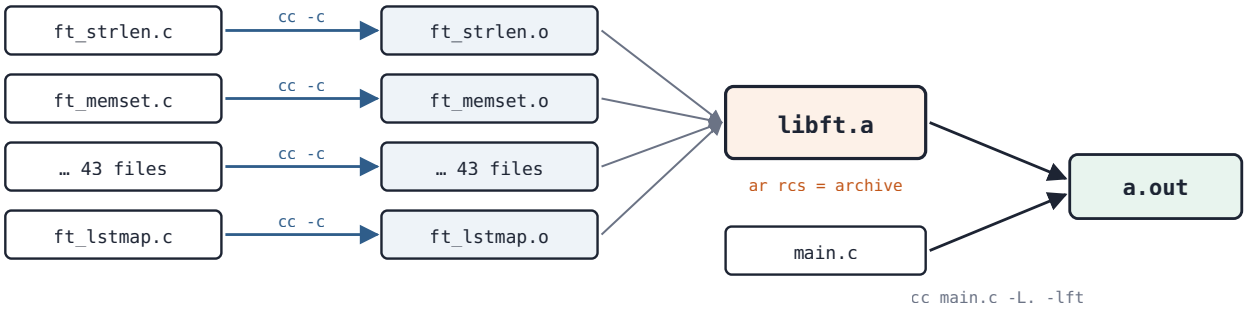


Figure — Compilation, archiving and linking

How to read a function page

- **Header card** — name, file, functions it calls, a category tag, and whether it calls `malloc` (if it does, the *caller* owns the memory and must `free` it).
- **Prototype table** — every parameter and its type, explained.
- **Mental model** — the idea in plain words, usually with a diagram of memory.
- **The code** — copied from the repository. Highlighted lines are the ones worth talking about.
- **Trace** — the values of variables, iteration by iteration, on a concrete input.
- **Edge cases & questions** — what the evaluator will try, and short, correct answers.

CHAPTER 2

C Foundations for Libft

> *Every question in a Libft defense comes back to a handful of ideas: bytes, types, pointers, the terminating \0, who owns heap memory, and how to read a function-pointer declaration. Master these and every function in the library becomes obvious.*

1. Where data lives

A running C program sees memory as one long array of bytes, each with an address. The operating system divides it into regions. You need four of them for Libft.

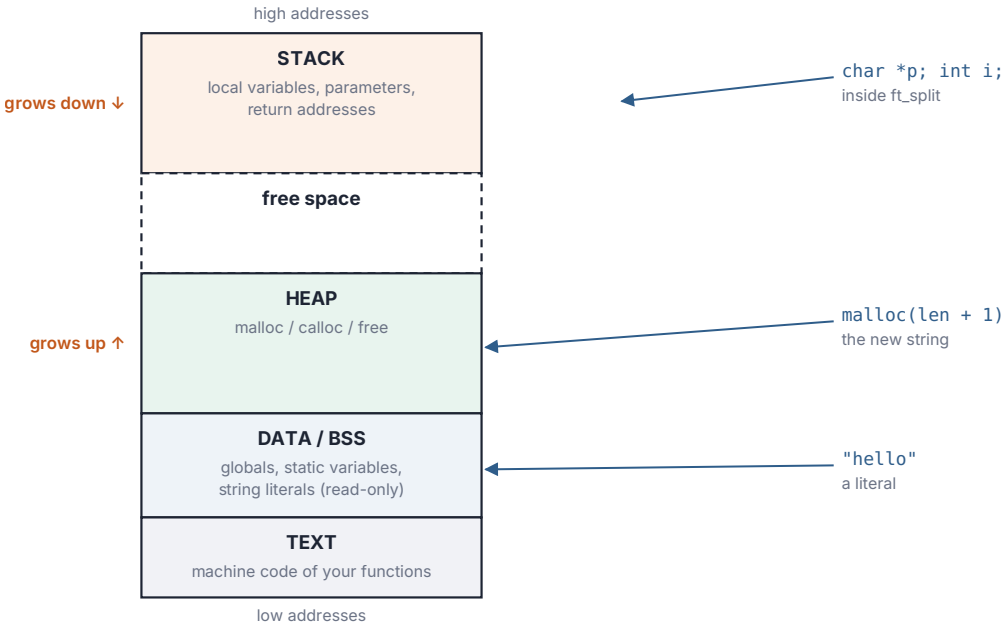


Figure — The memory layout of a process

REGION	LIFETIME	LIBFT EXAMPLE
Stack	Automatic: created when the function is called, destroyed when it returns.	`size_t i;` in ft_strlen. Returning a pointer to a local array is a bug — the memory is gone.
Heap	Manual: exists from `malloc` until `free`. Survives the function that created it.	Everything ft_strdup, ft_substr, ft_split, ft_itoa, ft_lstnew return. The *caller* must free it.
Data	Whole program.	String literals like `"-2147483648"` in ft_putnbr_fd.
Text	Whole program.	The compiled code of every function — what a *function pointer* points to.

KEY IDEA

The single most important rule of Libft: **if a function calls malloc, it returns ownership**. Whoever receives the pointer is responsible for `free`-ing it exactly once. And if malloc fails mid-way (ft_split, ft_lstmap), the function must free everything it already allocated before returning `NULL` — otherwise it leaks.

2. The data types used in this library

Sizes below are for the 64-bit Linux/macOS machines at 42 (LP64 model). The C standard only guarantees minimums; never hard-code these sizes, use `sizeof`.

TYPE	BYTES	RANGE	WHERE AND WHY IT IS USED IN LIBFT
`char`	1	−128…127 *or* 0…255 (implementation-defined!)	Characters of a string. Whether plain `char` is signed depends on the platform — signed on x86, unsigned on ARM Linux.
`unsigned char`	1	0…255	Raw bytes. All `mem*` functions and comparisons cast to it so that bytes ≥ 128 compare correctly.
`int`	4	−2 147 483 648 … 2 147 483 647	Characters passed to `is*` / `to*` (to allow `EOF` = −1), return values of compare functions, `fd`.
`unsigned int`	4	0 … 4 294 967 295	Index passed to the callbacks of strmapi/striteri; `start` in substr; return of ft_lstsize here.
`long`	8	$\pm 9.2 \cdot 10^{18}$	ft_itoa copies `n` into a `long` so that `-INT_MIN` does not overflow.
`long long`	8	±9 223 372 036 854 775 807	ft_atoi accumulates the result in it to detect overflow.
`size_t`	8	0 … 18 446 744 073 709 551 615	Sizes and lengths. Unsigned, as large as the biggest possible object. Returned by `sizeof` and strlen; taken by malloc.
`void *`	8	an address	“Pointer to anything”. Memory functions and t_list content use it so they work on any type.
`t_list`	16	—	Our struct: two pointers (8 + 8).

Why the casts to unsigned char?

A byte is just 8 bits. Whether those bits mean 233 or −23 depends on the type through which you read them.

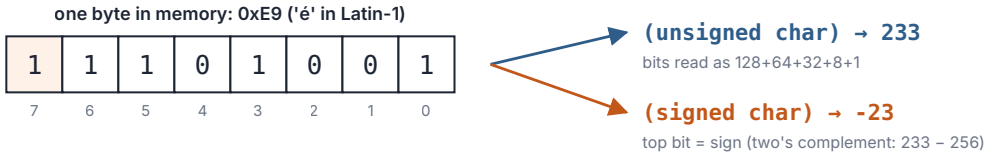


Figure — The same byte read as unsigned and as signed

That is why `ft_memcmp`, `ft_strncmp` and `ft_memchr` convert to `unsigned char`: the real libc compares bytes as unsigned values, so `"\xE9"` must be *greater* than `"a"`, not smaller. It is also why the C standard says the `int c` given to `memset`, `memchr`, `strchr` “is converted to `unsigned char`” (or to `char` for strchr).

> **IN THIS REPO**
>
> `ft_strchr` and `ft_strrchr` in this repository compare a plain (signed) `char` against `(unsigned char)c`. On x86 the byte 0xE9 is −23 on the left and 233 on the right, so they never match: `ft_strchr("café", 0xE9)` returns `NULL` while libc finds the character. Verified by test. Fix: compare with `(char)c`. See Chapter 11.

size_t and the unsigned trap

`size_t` is unsigned: it can never be negative. Subtraction that would go below zero **wraps around** to a huge number. `size - 1` when `size == 0` is 18 446 744 073 709 551 615. This is why ft_strlcpy and ft_strlcat test `size == 0` *before* computing `size - 1`, and why a loop like `while (n-- > 0)` is safe but `for (i = len - 1; i >= 0; i--)` with an unsigned `i` is an infinite loop.

const

`const char *s` means "I promise not to modify the characters `s` points to." The pointer itself may move (`s++` is fine). Read the declaration right-to-left: `const char *p` → pointer to const char; `char *const p` → const pointer to char. Functions like ft_strchr take `const char *` but return `char *` — they must cast the const away, exactly like libc does, so the caller can modify the string they own.

3. Pointers, arrays and strings

A pointer is a variable that stores an address. `*p` reads what is at that address; `p[i]` is exactly `*(p + i)`. Pointer arithmetic moves by **elements, not bytes**: `p + 1` advances 1 byte for a `char *`, 4 for an `int *`, 16 for a `t_list *`. That is the reason you cannot do arithmetic on a `void *` (element size unknown) — every memory function first copies it into an `unsigned char *`.

> DEFINITION
>
> A **C string** is a sequence of `char` ending with the byte `'\0'` (value 0). There is no length stored anywhere: every string function finds the end by walking until it meets that zero. A buffer for a string of length n needs **n + 1** bytes. Every `malloc(len + 1)` in this library is that rule.

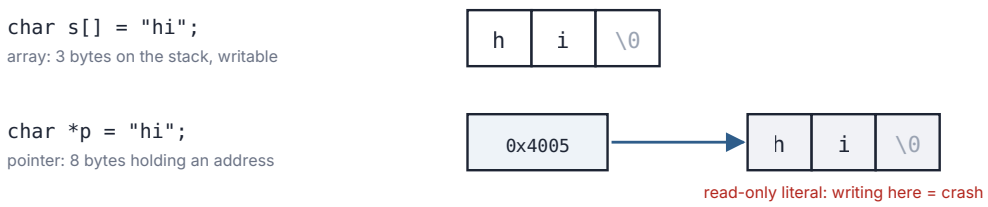


Figure — An array owns its bytes; a pointer only refers to bytes elsewhere

EXPRESSION	MEANS	VALUE FOR CHAR S[] = "HI"
`s`	address of first element (decays to `char *`)	&s[0]
`*s` / `s[0]`	first character	'h'
`s[2]`	the terminator	'\0' (0)
`sizeof(s)`	size of the array	3
`sizeof(p)` for `char *p`	size of a pointer	8
`&s[i]`	address of element i — what strchr returns	s + i

NULL vs '\0' vs ""

	TYPE	MEANING
`NULL`	pointer (`(void *)0`)	Points nowhere. Dereferencing it crashes (segfault). malloc returns it on failure.
`'\0'`	char (value 0)	The terminator byte at the end of a string.
`""`	`char *` to a 1-byte array	A valid empty string: one byte, which is `'\0'`. ft_strtrim returns a heap copy of it.

4. malloc and free — the contract

THE TWO ALLOWED ALLOCATION FUNCTIONS

```
void    *malloc(size_t size);   /* returns uninitialised bytes, or NULL */
void    free(void *ptr);        /* free(NULL) is allowed and does nothing */
```

- Always check the return value: `if (!ptr) return (NULL);`
- malloc'd memory contains garbage — that is the difference with calloc, which zeroes it.
- Free exactly once. Twice = *double free* (crash/undefined). Never = *leak*.
- Never use a pointer after freeing it (*use after free*).
- To check for leaks: `valgrind --leak-check=full ./a.out` on Linux, `leaks -atExit -- ./a.out` on macOS.

5. Function pointers

A function's code lives in memory too (the text segment), so it has an address. A function pointer stores that address, and calling through it runs the function. This is how ft_strmapi, ft_striteri, ft_lstiter, ft_lstmap and ft_lstdelone can do "something" to every element without knowing what — the caller decides. Functions that take other functions are called **higher-order** functions.



Read from the name outward: f → (*f) pointer → (...) to function → char returning char

Figure — Decoding the callback parameter of ft_strmapi

USING A CALLBACK

```
1  char    to_upper_at(unsigned int i, char c)    /* matches char (*)(unsigned int, char) */
2  {
3      (void)i;
4      return (ft_toupper(c));
5  }
6
7  char    *s = ft_strmapi("hello", to_upper_at);   /* pass the NAME: it decays to a pointer */
8  /* inside strmapi:  str[i] = f(i, s[i]);   calls to_upper_at(i, s[i]) */
```

DECLARATION IN LIBFT.H	F TAKES	F RETURNS
`char (*f)(unsigned int, char)`	index, character (by value)	the new character
`void (*f)(unsigned int, char *)`	index, address of the character	nothing — modifies in place
`void (*f)(void *)`	a node's content	nothing
`void *(*f)(void *)`	a node's content	new content (a pointer)
`void (*del)(void *)`	a node's content	nothing — frees it

6. static helpers, headers and include guards

A function declared `static` at file level is visible **only inside that .c file**. ft_atoi's `issign`, ft_split's `w_count`, ft_itoa's `ft_intlen`, ft_substr's `guard` are static. Two different files can each have a static function with the same name without a link conflict, and helpers do not pollute the library's public interface. The Norm allows at most 5 functions per file, which is why helpers are split this way.

A header (`.h`) contains **declarations** (prototypes, types), not definitions. `#include "libft.h"` literally pastes the header in. Without the prototype, the compiler does not know a function's parameter or return types — with `-Werror`, "implicit declaration of function" stops the build. The `#ifndef LIBFT_H / # define LIBFT_H / #endif` **include guard** makes a second inclusion expand to nothing, preventing "redefinition of t_list" errors.

7. File descriptors and write

A file descriptor is a small `int` the kernel gives you as a handle to an open file, terminal, pipe or socket. Three are open from the start: **0** standard input, **1** standard output, **2** standard error. `open()` returns new ones (3, 4…).

THE ONLY OUTPUT PRIMITIVE ALLOWED

```
ssize_t write(int fd, const void *buf, size_t count);  /* from <unistd.h> */
```

`write` sends `count` bytes starting at `buf` to `fd` and returns how many were written (or −1). It does not care about `'\0'` — that is why ft_putstr_fd computes the length first. `printf` is buffered; `write` is not, so when you mix them output can appear out of order.

8. Integer limits and INT_MIN

In two's complement, `int` ranges from −2 147 483 648 to 2 147 483 647: there is **one more negative number than positive**. So `-(-2147483648)` does not fit in an int — signed overflow is undefined behaviour. Every function that negates a number has to deal with this:

FUNCTION	STRATEGY USED IN THIS REPO
ft_itoa	Copy `n` into a `long` first; negating a long holding INT_MIN is fine.
ft_putnbr_fd	Special case: if `n == -2147483648`, print the literal string and return.
ft_atoi	Accumulate in `long long` and check before multiplying, mirroring glibc's behaviour on overflow.

QUESTIONS YOU MAY BE ASKED

Q What is the difference between the stack and the heap?

→ Stack memory is automatic and freed when the function returns; heap memory comes from malloc and lives until free. Functions that return new strings must use the heap.

Q Why does strlen return size_t and not int?

→ A length can never be negative and an object can be larger than INT_MAX bytes; size_t is the unsigned type big enough to hold the size of any object.

Q Why do you cast void * to unsigned char * in the mem functions?

→ You cannot dereference or do arithmetic on void * (unknown element size). unsigned char * lets you walk byte by byte, and reading bytes as unsigned gives the 0–255 values the standard requires for comparisons.

Q What happens if you call free twice on the same pointer?

→ Undefined behaviour: usually an abort ("double free detected") or heap corruption.

Q Why do is* functions take an int and not a char?

→ So they can receive every unsigned char value plus EOF (–1), the values returned by getchar().

CHAPTER 3

libft.h, Line by Line

> *The header is the contract of the library: the one struct, the 43 prototypes, and the include guard.*

LIBFT.H · AS WRITTEN IN THE REPOSITORY

```
#ifndef LIBFT_H
# define LIBFT_H
# include <stddef.h>
# include <stdlib.h>
typedef struct s_list
{
    void            *content;
    struct s_list   *next;
}                   t_list;

int                 ft_atoi(const char *str);
int                 ft_toupper(int c);
int                 ft_tolower(int c);
int                 ft_strncmp(const char *s1, const char *s2, size_t n);
int                 ft_memcmp(const void *s1, const void *s2, size_t n);
int                 ft_memcmp(const void *s1, const void *s2, size_t n);
int                 ft_isprint(int c);
int                 ft_isdigit(int c);
int                 ft_isascii(int c);
int                 ft_isalpha(int c);
int                 ft_isalnum(int c);
unsigned int        ft_lstsize(t_list *lst);
size_t              ft_strlen(char const *s);
size_t              ft_strlcpy(char *dest, const char *src, size_t size);
size_t              ft_strlcat(char *dest, const char *src, size_t size);
char                *ft_substr(char const *s, unsigned int start, size_t len);
char                *ft_strtrim(char const *s1, char const *set);
char                *ft_strchr(const char *s, int c);
char                *ft_strrchr(char const *s, int c);
char                *ft_strnstr(const char *haystack, const char *needle,
                        size_t len);
char                *ft_strmapi(char const *s, char (*f)(unsigned int, char));
char                *ft_strdup(const char *s);
char                *ft_itoa(int n);
char                **ft_split(char const *s, char c);
void                ft_striteri(char *s, void (*f)(unsigned int, char *));
void                ft_putchar_fd(char c, int fd);
void                ft_putstr_fd(char *s, int fd);
void                ft_putnbr_fd(int n, int fd);
void                ft_putendl_fd(char *s, int fd);
void                ft_bzero(void *s, size_t n);
void                *ft_memset(void *s, int c, size_t len);
void                *ft_memchr(const void *s, int c, size_t n);
void                *ft_memmove(void *dest, const void *src, size_t n);
void                *ft_memcpy(void *dest, const void *src, size_t n);
void                *ft_calloc(size_t count, size_t size);
void                ft_lstiter(t_list *lst, void (*f)(void *));
void                ft_lstdelone(t_list *lst, void (*del)(void *));
void                ft_lstclear(t_list **lst, void (*del)(void *));
void                ft_lstadd_front(t_list **lst, t_list *new);
```

```
51 void                ft_lstadd_back(t_list **lst, t_list *new);
52 t_list              *ft_lstnew(void *content);
53 t_list              *ft_lstmap(t_list *lst, void *(*f)(void *),
54                          void (*del)(void *));
55 t_list              *ft_lstlast(t_list *lst);
56 #endif
```

The include guard (lines 1–2 and the last line)

The first time the preprocessor sees the file, `LIBFT_H` is undefined, so it defines it and keeps the content. Any later `#include "libft.h"` in the same translation unit finds `LIBFT_H` already defined and skips everything until `#endif`. The Norm wants the guard name to be the file name in upper case, and the space after `#` inside it (`# define`) is the Norm's indentation of preprocessor directives.

The system includes

HEADER	BRINGS IN	NEEDED FOR
`<stddef.h>`	`size_t`, `NULL`	almost every prototype
`<stdlib.h>`	`malloc`, `free`	every allocating function, ft_lstdelone
(missing) `<unistd.h>`	`write`	ft_putstr_fd includes `<fcntl.h>` instead, which does not declare write — another compile error (Chapter 11).

The struct: typedef struct s_list

THE ONLY STRUCT IN THE LIBRARY

```
1 typedef struct s_list
2 {
3     void            *content;   /* address of the data this node carries */
4     struct s_list   *next;      /* address of the following node, or NULL */
5 }                   t_list;
```

Two things happen in this one statement:

1. `struct s_list { … }` defines a **structure type** named `struct s_list` with two members.
2. `typedef … t_list;` creates an **alias**, so you can write `t_list` instead of `struct s_list`. The Norm requires struct names to start with `s_` and typedefs with `t_`.

> KEY IDEA
>
> **Why `struct s_list *next` and not `t_list *next`?** Inside the braces the typedef does not exist yet — `t_list` is only created at the closing `;`. The struct tag `s_list`, however, is known from the first line. A struct may contain a **pointer** to its own type (fixed size: 8 bytes), never a member of its own type (which would be infinitely large). This is called a **self-referential struct**.

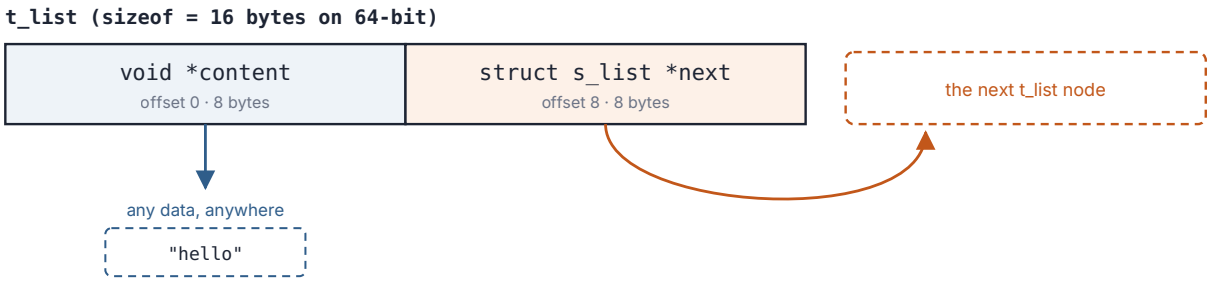


Figure — Memory layout of one t_list node

Accessing members: . versus →

YOU HAVE	SYNTAX	EXAMPLE
a struct value	`node.content`	`t_list n; n.next = NULL;`
a pointer to a struct	`ptr->content` ≡ `(*ptr).content`	`t_list *n = ft_lstnew(x); n->next = NULL;`
a pointer to a pointer	`(*lst)->next`	used in ft_lstadd_front / ft_lstclear on `t_list **lst`

Why is `content` a `void *`? Because the list should hold anything: strings, ints, other structs. The list never looks inside `content`; it just stores the address. The functions that need to understand the data — free it, transform it — receive a function pointer from the user (`del`, `f`). Chapter 9 covers the list functions in depth.

The prototypes

Each line tells the compiler a function's name, return type and parameter types, so any file that includes the header can call it. Alignment with tabs is a Norm rule. Notice in this file:

IN THIS REPO

- `ft_memcmp` is declared **twice** (lines 15–16). Legal C (identical declarations are allowed) but sloppy — an evaluator will point it out.
- `ft_strjoin` is **not declared at all**, so any program that calls it gets an implicit-declaration error.
- `ft_lstsize` returns `unsigned int`; the subject's prototype is `int ft_lstsize(t_list *lst)`.
- `<unistd.h>` is not included anywhere for `write`.

QUESTIONS YOU MAY BE ASKED

Q What is the difference between struct s_list and t_list?
→ Same type. s_list is the struct tag; t_list is a typedef alias created by the typedef so you do not have to write the word struct.

Q Why can't the struct contain a t_list instead of a pointer?
→ A struct containing itself would have infinite size. A pointer has a fixed size (8 bytes), so a self-referential pointer is fine.

Q What would happen without the include guard?
→ If libft.h were included twice in one file (directly or through another header), t_list would be defined twice → compile error.

Q What is sizeof(t_list)?
→ 16 on a 64-bit machine: two 8-byte pointers, no padding.

Q Why are system headers in angle brackets and libft.h in quotes?
→ Quotes search the current directory first, then the system paths; angle brackets search only the system include paths.

CHAPTER 4

Character Functions

> *Seven tiny functions that classify or convert one character. They are the easiest in the library — which is exactly why evaluators use them to test whether you know ASCII and why the parameter is an int.*

All seven take an `int c`. In libc, these functions are defined for every value an `unsigned char` can hold **plus EOF** (−1) — the possible return values of `getchar()`. Using `int` allows that extra value. They return **non-zero for true** and 0 for false; libc often returns values like 1024 or 8, so tests should compare to 0, never to 1. This repo returns exactly 1, which is allowed.

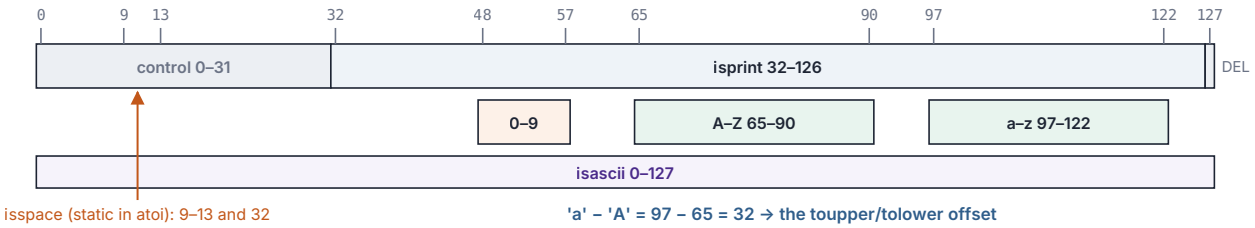


Figure — The ASCII ranges every character function tests

> TIP
>
> Comparing with character literals (`c >= 'a'`) is clearer than magic numbers (`c >= 97`) and means the same thing: `'a'` is just the int 97 in C. ft_isascii and ft_isprint in this repo use numbers; both are acceptable.

ft_isalpha

ft_isalpha.c

CHARACTER · NO ALLOCATION · MIRRORS LIBC ISALPHA

```c
int         ft_isalpha(int c);
```

True if `c` is an English letter, upper or lower case. Two ranges, joined with `||`.

FT_ISALPHA.C · AS WRITTEN IN THE REPOSITORY

```c
int ft_isalpha(int c)
{
    if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z'))
        return (1);
    return (0);
}
```

INPUT	TEST	RESULT
`'G'` (71)	65 ≤ 71 ≤ 90	1

INPUT	TEST	RESULT
`'z'` (122)	97 ≤ 122 ≤ 122	1
`'['` (91)	between the ranges	0
`'é'` (233)	above 127	0
`-1` (EOF)	below 0	0

ft_isdigit

ft_isdigit.c

CHARACTER NO ALLOCATION MIRRORS LIBC ISDIGIT

```
int         ft_isdigit(int c);
```

True for `'0'`…`'9'` (48–57). Note: **the character** '5', not the integer 5. `ft_isdigit(5)` is 0. Used by ft_atoi and ft_isalnum.

FT_ISDIGIT.C · AS WRITTEN IN THE REPOSITORY

```c
int ft_isdigit(int c)
{
    if (c >= '0' && c <= '9')
        return (1);
    return (0);
}
```

ft_isalnum

ft_isalnum.c · uses `ft_isalpha`, `ft_isdigit`

CHARACTER NO ALLOCATION MIRRORS LIBC ISALNUM

```
int         ft_isalnum(int c);
```

Letter *or* digit. Reuses the two previous functions instead of repeating the ranges — good practice, and evaluators like seeing reuse. `||` short-circuits: if `ft_isalpha` returns true, `ft_isdigit` is not called. The expression itself evaluates to 0 or 1.

FT_ISALNUM.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"

int ft_isalnum(int c)
{
    return (ft_isalpha(c) || ft_isdigit(c));
}
```

ft_isascii

ft_isascii.c

CHARACTER NO ALLOCATION MIRRORS LIBC ISASCII

```
int         ft_isascii(int c);
```

True for 0–127: the 7-bit ASCII set. 128–255 are “extended” bytes (Latin-1, UTF-8 continuation bytes…), not ASCII. 0 itself (`'\0'`) **is** ASCII.

FT_ISASCII.C · AS WRITTEN IN THE REPOSITORY

```c
int ft_isascii(int c)
{
    if (c >= 0 && c <= 127)
        return (1);
    return (0);
}
```

ft_isprint

ft_isprint.c

CHARACTER NO ALLOCATION MIRRORS LIBC ISPRINT

```
int         ft_isprint(int c);
```

True for characters that occupy a printing position: 32 (space) to 126 (`~`). Space is printable; tab (9) and newline (10) are not; DEL (127) is not.

FT_ISPRINT.C · AS WRITTEN IN THE REPOSITORY

```c
int ft_isprint(int c)
{
    if (c >= 32 && c <= 126)
        return (1);
    return (0);
}
```

ft_toupper

ft_toupper.c

CONVERSION NO ALLOCATION MIRRORS LIBC TOUPPER

```
int         ft_toupper(int c);
```

If `c` is a lowercase letter, return its uppercase version; otherwise return `c` **unchanged**. In ASCII, each uppercase letter is exactly 32 below its lowercase one (bit 5 is the only difference: `'a'` = 0110 0001, `'A'` = 0100 0001).

FT_TOUPPER.C · AS WRITTEN IN THE REPOSITORY

```
1  int ft_toupper(int c)
2  {
3      if (c >= 'a' && c <= 'z')
4          return (c - 32);
5      return (c);
6  }
```

ft_tolower

ft_tolower.c

CONVERSION NO ALLOCATION MIRRORS LIBC TOLOWER

```
int         ft_tolower(int c);
```

The mirror image: `'A'`…`'Z'` → add 32. Anything else, including digits and already-lowercase letters, comes back unchanged.

FT_TOLOWER.C · AS WRITTEN IN THE REPOSITORY

```
1  int ft_tolower(int c)
2  {
3      if (c >= 'A' && c <= 'Z')
4          return (c + 32);
5      return (c);
6  }
```

TRACE — CONVERSIONS

CALL	IN RANGE?	RETURNS
ft_toupper('q')	113 in 97–122 ✓	113 − 32 = 81 = 'Q'
ft_toupper('Q')	81 not lowercase	81 = 'Q' (unchanged)
ft_toupper('4')	52 ✗	'4'
ft_tolower('M')	77 in 65–90 ✓	77 + 32 = 109 = 'm'
ft_tolower(-1)	✗	−1 (EOF passes through)

TIP

Equivalent forms you may be asked to write on the spot: `c - ('a' - 'A')`, or with bits `c & ~32` (to upper) and `c | 32` (to lower) — but the bit trick only works *after* checking the range.

QUESTIONS YOU MAY BE ASKED

Q Why does ft_toupper return int and not char?
→ To match libc and to be able to return EOF (−1) unchanged; an int can represent every unsigned char value plus EOF.

Q What does ft_isalpha('5') return? And ft_isdigit(5)?
→ 0 and 0. '5' is not a letter; 5 is the control character ENQ, not the digit character '5' (53).

Q Is the space printable?
→ Yes, 32 is in the 32–126 range. Tab and newline are not.

Q Why would ft_isalpha(233) be 0 here but maybe non-zero in libc?
→ libc depends on the locale; in a Latin-1 locale 233 ('é') is a letter. In the default "C" locale it is not, which is what we mirror.

Q Could you write ft_isalnum without calling the other two?
→ Yes, with three ranges in one condition, but reusing functions avoids duplicated logic and bugs.

CHAPTER 5

Memory Functions

> *The mem* family works on raw bytes, not strings: they do not stop at \0, they take an explicit byte count n, and they accept any type through void *. The star of the chapter is ft_memmove and its overlap logic.*

FUNCTION	PURPOSE	RETURNS
ft_memset	fill n bytes with a value	dest
ft_bzero	fill n bytes with 0	nothing
ft_memcpy	copy n bytes, regions must not overlap	dest
ft_memmove	copy n bytes, overlap allowed	dest
ft_memchr	find first byte equal to c in n bytes	pointer or NULL
ft_memcmp	compare n bytes	<0, 0, >0
ft_calloc	allocate count×size zeroed bytes	pointer or NULL

KEY IDEA

Pattern shared by every function here: (1) copy the `void *` parameter into an `unsigned char *` local, (2) loop on an index `i` from 0 to `n`, (3) work one byte at a time. Never test for `'\0'` — a 0 byte is ordinary data here.

ft_memset

ft_memset.c

MEMORY · NO ALLOCATION · MIRRORS LIBC MEMSET

```
void        *ft_memset(void *s, int c, size_t len);
```

PARAMETER / RETURN	TYPE	MEANING
`s`	`void *`	start of the memory to fill (any type)
`c`	`int`	the value; converted to unsigned char, so only the low 8 bits count (256 → 0)
`len`	`size_t`	number of bytes to fill
`return`	`void *`	the original s, to allow chaining

Writes the same byte `len` times. Key subtlety: memset fills **bytes**, not elements. `ft_memset(arr, 1, 4 * sizeof(int))` on an int array does *not* give 1s — each int becomes `0x01010101` = 16 843 009. It only "works" for 0 and −1 (all bits 0 or all bits 1).

FT_MEMSET.C · AS WRITTEN IN THE REPOSITORY

```c
void    *ft_memset(void *s, int c, size_t len)
{
    unsigned char   *p;
    size_t          i;

    p = s;
    i = 0;
    while (i < len)
        p[i++] = (unsigned char)c;
    return (s);
}
```

`p = s;` needs no cast: in C a `void *` converts implicitly to any object pointer. `p[i++] = …` writes at index `i`, *then* increments it (post-increment).

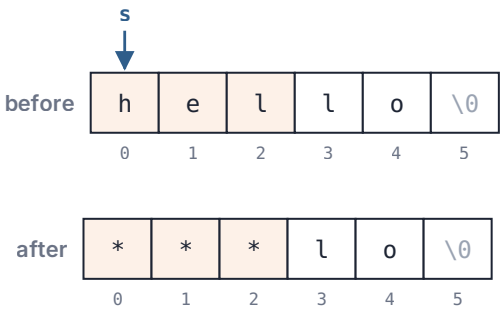


Figure — ft_memset(s, '*', 3) — returns s

ft_bzero

ft_bzero.c · uses `ft_memset`

MEMORY · NO ALLOCATION · MIRRORS LIBC BZERO

```c
void        ft_bzero(void *s, size_t n);
```

Writes `n` zero bytes. It is simply `memset(s, 0, n)` — and that is how this repo writes it. bzero is a BSD function (deprecated by POSIX in favour of memset) and returns nothing.

FT_BZERO.C · AS WRITTEN IN THE REPOSITORY

```c
#include <ctype.h>
#include <string.h>

void    ft_bzero(void *s, size_t n)
{
    ft_memset(s, 0, n);
}
```

IN THIS REPO

This file includes `<ctype.h>` and `<string.h>` (useless) but not `"libft.h"`, so `ft_memset` and `size_t` are undeclared → the build fails with `-Werror`.

ft_memcpy

ft_memcpy.c

MEMORY NO ALLOCATION MIRRORS LIBC MEMCPY

```
void        *ft_memcpy(void *dest, const void *src, size_t n);
```

PARAMETER / RETURN	TYPE	MEANING
`dest`	`void *`	where to write
`src`	`const void *`	where to read (not modified → const)
`n`	`size_t`	number of bytes
`return`	`void *`	dest

Copies `n` bytes from `src` to `dest`, front to back. The standard says the behaviour is **undefined if the regions overlap** — that freedom lets real implementations copy 8 or 16 bytes at a time. If they might overlap, use memmove.

FT_MEMCPY.C · AS WRITTEN IN THE REPOSITORY

```
1  void    *ft_memcpy(void *dest, const void *src, size_t n)
2  {
3      unsigned char       *d;
4      const unsigned char *s;
5      size_t              i;
6
7      d = (unsigned char *)dest;
8      s = (const unsigned char *)src;
9      i = 0;
10     while (i < n)
11     {
12         d[i] = s[i];
13         i++;
14     }
15     return (dest);
16 }
```

PITFALL

Passing NULL to memcpy is undefined behaviour in the standard. Some testers (francinette, libftTester) nevertheless call `ft_memcpy(NULL, NULL, 3)` and expect NULL back, because Apple's libc returns early when `dest == src`. Many students add `if (!dest && !src) return (dest);` for that reason. This repo has no such check, so that call would segfault. Both behaviours are defensible — know which one you chose and why.

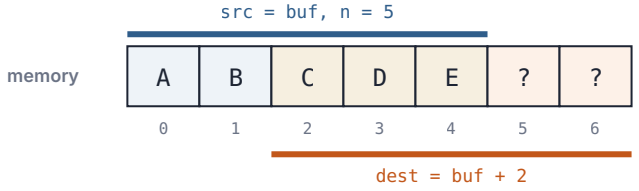
ft_memmove

ft_memmove.c

MEMORY · NO ALLOCATION · MIRRORS LIBC MEMMOVE

```
void        *ft_memmove(void *dest, const void *src, size_t n);
```

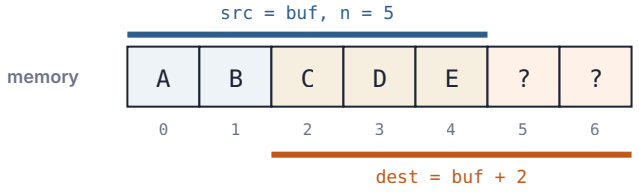
Same as memcpy, but correct even when the two regions overlap. The trick: choose the copy **direction** so that you never read a byte you have already overwritten.



Forward copy (memcpy): WRONG

```
i=0: d[0]=s[0] → A B A D E ? ?
i=1: d[1]=s[1] → A B A B E ? ?
i=2: d[2]=s[2] → A B A B A ? ?
↑ s[2] was already overwritten
result: A B A B A B A
```

Figure — Shifting "ABCDE" two bytes right with a forward copy destroys the source



Backward copy (memmove): RIGHT

```
n=4: d[4]=s[4] → A B C D E ? E
n=3: d[3]=s[3] → A B C D E D E
n=2: d[2]=s[2] → A B C D C D E
n=1, n=0 …
result: A B A B C D E ✓
```

Figure — The same move copied from the end: every byte is read before it is overwritten

SITUATION	DIRECTION	WHY
`dest > src` (dest after src)	back to front	the end of src lies inside dest; copy it first before it gets overwritten
`dest < src` (dest before src)	front to back	the start of src lies inside dest — reading forward stays ahead of the writes
`dest == src` or `n == 0`	nothing	nothing to do; return immediately

FT_MEMMOVE.C · AS WRITTEN IN THE REPOSITORY

```
1  void    *ft_memmove(void *dest, const void *src, size_t n)
2  {
3      unsigned char       *d;
4      const unsigned char *s;
5
6      d = dest;
7      s = src;
8      if (d == s || n == 0)
9          return (dest);
10     if (d > s)
11     {
12         while (n--)
13             d[n] = s[n];
14     }
15     else
16     {
```

```
17          while (n--)
18              *d++ = *s++;
19      }
20      return (dest);
21  }
```

Line 12 `while (n--)`: tests `n`, *then* decrements. With n = 5 the body runs with n = 4, 3, 2, 1, 0 — exactly the valid indexes from the last down to the first — and the loop stops when the test sees 0. Because n is unsigned, after the last test it wraps to SIZE_MAX, but it is never used again. **Line 18** `*d++ = *s++`: copy the byte, then advance both pointers.

TRACE — FT_MEMMOVE(BUF + 2, BUF, 5)

N TESTED	N IN BODY	OPERATION	BUFFER (INDEXES 0–6)
5	4	d[4] = s[4] → buf[6] = 'E'	A B C D E ? E
4	3	d[3] = s[3] → buf[5] = 'D'	A B C D E D E
3	2	d[2] = s[2] → buf[4] = 'C'	A B C D C D E
2	1	d[1] = s[1] → buf[3] = 'B'	A B C B C D E
1	0	d[0] = s[0] → buf[2] = 'A'	A B A B C D E
0	–	loop ends	A B A B C D E ✓

EVALUATOR ASKS

"Comparing two pointers with `>` — is that legal?" Strictly, the standard only defines relational comparison for pointers into the same object; for memmove that is exactly the case where it matters (overlap). When they are unrelated, either direction is correct anyway. Every common libc does the same comparison.

ft_memchr

ft_memchr.c

MEMORY · NO ALLOCATION · MIRRORS LIBC MEMCHR

```
void        *ft_memchr(const void *s, int c, size_t n);
```

Scans the first `n` bytes of `s` for the byte `(unsigned char)c`. Returns its address or `NULL`. Unlike strchr it does **not** stop at `'\0'` — it can find a 0 byte, or bytes after it.

FT_MEMCHR.C · AS WRITTEN IN THE REPOSITORY

```
1  void    *ft_memchr(const void *s, int c, size_t n)
2  {
3      const unsigned char *ptr;
4      size_t              i;
5
6      ptr = (const unsigned char *)s;
7      i = 0;
8      while (i < n)
9      {
10         if (ptr[i] == (unsigned char)c)
11             return ((void *)&ptr[i]);
12         i++;
13     }
```

```
14      return (NULL);
15  }
```

The cast `(void *)&ptr[i]` both converts the type and drops `const`: the input was `const void *` but the prototype returns `void *`. Same design as libc.

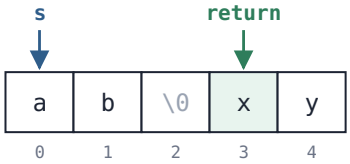


Figure — ft_memchr(s, 'x', 5) walks past the \0 and finds 'x' at index 3; with n = 3 it would return NULL

ft_memcmp

ft_memcmp.c

MEMORY NO ALLOCATION MIRRORS LIBC MEMCMP

```
int         ft_memcmp(const void *s1, const void *s2, size_t n);
```

Compares byte by byte until a difference or `n` bytes. Returns the difference of the first differing pair, read as **unsigned char**; 0 if all n bytes are equal. Only the sign is guaranteed by the standard, not the exact value.

FT_MEMCMP.C · AS WRITTEN IN THE REPOSITORY

```
1  int ft_memcmp(const void *s1, const void *s2, size_t n)
2  {
3      const unsigned char *p1;
4      const unsigned char *p2;
5      size_t              i;
6
7      if (n == 0)
8          return (0);
9      p1 = (const unsigned char *)s1;
10     p2 = (const unsigned char *)s2;
11     i = 0;
12     while (i < n && (*p1 == *p2))
13     {
14         p1++;
15         p2++;
16         i++;
17     }
18     if (i == n)
19         return (0);
20     return (*p1 - *p2);
21 }
```

Why the `if (i == n)` check? When the loop stops because `i` reached `n`, `p1` and `p2` point one past the compared zone: dereferencing them would read bytes that are not part of the comparison (maybe out of bounds). `*p1 - *p2`: both are promoted to `int` before subtracting, so 0 − 255 gives −255, not a wrapped unsigned value.

TRACE — FT_MEMCMP("ABCD", "ABZD", 4)

I	*P1	*P2	EQUAL?
0	'a' 97	'a' 97	yes → advance
1	'b' 98	'b' 98	yes → advance
2	'c' 99	'z' 122	no → loop stops
			i (2) ≠ n (4) → return 99 − 122 = −23

ft_calloc

ft_calloc.c · uses `ft_bzero`

ALLOCATING MEMORY | ALLOCATES (MALLOC) | MIRRORS LIBC CALLOC

```
void        *ft_calloc(size_t count, size_t size);
```

PARAMETER / RETURN	TYPE	MEANING
`count`	`size_t`	number of elements
`size`	`size_t`	size of one element in bytes
`return`	`void *`	zero-filled block of count × size bytes, or NULL

calloc = malloc + zeroing + **overflow protection**. Three jobs, three blocks of code:

FT_CALLOC.C · AS WRITTEN IN THE REPOSITORY

```
 1  #include <stdint.h>
 2
 3  void    *ft_calloc(size_t count, size_t size)
 4  {
 5      size_t          len;
 6      unsigned char   *ptr;
 7
 8      if (size != 0 && count > SIZE_MAX / size)
 9          return (NULL);
10      len = count * size;
11      if (!len)
12          len = 1;
13      ptr = malloc(len);
14      if (!ptr)
15          return (NULL);
16      ft_bzero(ptr, len);
17      return ((void *)ptr);
18  }
```

1 · THE OVERFLOW CHECK

`count * size` is computed in `size_t`; if the true product exceeds SIZE_MAX it silently wraps to a small number, malloc succeeds with a tiny block, and the caller writes far beyond it. The condition `count > SIZE_MAX / size` is the algebraic form of `count × size > SIZE_MAX` without multiplying. The `size != 0` guard prevents a division by zero. `SIZE_MAX` comes from `<stdint.h>`.

2 · ZERO-SIZE REQUESTS

`calloc(0, 5)` : the standard allows returning NULL *or* a unique pointer that can be freed. This repo allocates 1 byte, so the caller always gets a freeable non-NULL pointer (glibc does the same).

3 · ZEROING

malloc returns garbage; ft_bzero clears it.

TRACE — THE THREE BRANCHES

CALL	CHECK	RESULT
`ft_calloc(4, sizeof(int))`	`4 > SIZE_MAX/4? no`	`malloc(16), 16 zero bytes`
`ft_calloc(0, 8)`	`size≠0, 0 > …? no; len = 0 → 1`	`malloc(1), 1 zero byte`
`ft_calloc(SIZE_MAX, 2)`	`SIZE_MAX > SIZE_MAX/2 → yes`	`NULL (no allocation)`

IN THIS REPO

The file includes `<stdint.h>` but not `"libft.h"` : `size_t` , `malloc` and `ft_bzero` are unknown → compile error.

QUESTIONS YOU MAY BE ASKED

Q What is the difference between memcpy and memmove?
→ memmove handles overlapping regions by choosing the copy direction; memcpy's behaviour on overlap is undefined (but it can be faster).

Q Why doesn't memcpy check for '\0'?
→ It copies raw bytes of any type — ints, structs — where 0 is valid data. The length is given by n.

Q Why use unsigned char * and not char * in the mem functions?
→ To read each byte as 0–255; comparisons in memcmp must treat 0xFF as larger than 0x01, and char may be signed.

Q What does memset(arr, 1, sizeof(arr)) do to an int array?
→ Every int becomes 0x01010101 (16843009); memset sets bytes, not ints.

Q Why does calloc check count > SIZE_MAX / size?
→ To detect that count × size would overflow size_t without performing the overflowing multiplication.

Q Can memcmp return 42?
→ Yes. Only the sign is specified; this implementation returns the difference of the first differing bytes.

Q What does memmove return?
→ The original dest pointer, never NULL.

CHAPTER 6

String Functions

> *Measuring, copying, searching, comparing, duplicating and converting C strings. Two functions deserve the most attention: ft_strlcat, whose return value surprises everybody, and ft_atoi, whose overflow handling is unusual in this repo.*

ft_strlen

ft_strlen.c

STRING · NO ALLOCATION · MIRRORS LIBC STRLEN

```
size_t        ft_strlen(char const *s);
```

Counts the characters before the `'\0'`. The terminator is not counted. `char const *` and `const char *` are the same type (const applies to the char).

FT_STRLEN.C · AS WRITTEN IN THE REPOSITORY

```
1  #include "libft.h"
2
3  size_t  ft_strlen(char const *s)
4  {
5      size_t  i;
6
7      i = 0;
8      while (s[i])
9          i++;
10     return (i);
11 }
```

`while (s[i])` is shorthand for `while (s[i] != '\0')`: in C any non-zero value is true. Time is O(n) — the whole string must be walked, because the length is stored nowhere. That is why calling strlen inside a loop condition (`while (i < ft_strlen(s))`) turns an O(n) loop into O(n²).

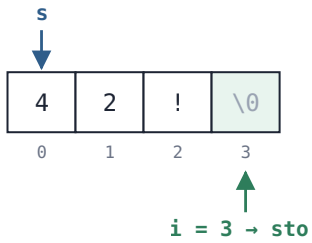


Figure — ft_strlen("42!") returns 3

PITFALL

`ft_strlen(NULL)` segfaults — exactly like the real strlen. Do not add a NULL check returning 0 unless you can defend it: it hides bugs in the caller.

ft_strlcpy

ft_strlcpy.c · uses `ft_strlen`

STRING | NO ALLOCATION | MIRRORS LIBC STRLCPY (BSD)

```
size_t        ft_strlcpy(char *dest, const char *src, size_t size);
```

PARAMETER / RETURN	TYPE	MEANING
`dest`	`char *`	destination buffer
`src`	`const char *`	string to copy
`size`	`size_t`	**total** size of dest in bytes, including room for '\0'
`return`	`size_t`	strlen(src) — the length it *tried* to create

A safe strcpy: copies at most `size - 1` characters and **always** terminates (if size > 0). Returning the length of `src` lets the caller detect truncation with one test: `if (ft_strlcpy(d, s, sz) >= sz) /* truncated */`.

FT_STRLCPY.C · AS WRITTEN IN THE REPOSITORY

```
1  #include "libft.h"
2
3  size_t  ft_strlcpy(char *dest, const char *src, size_t size)
4  {
5      size_t  slen;
6      size_t  i;
7
8      slen = ft_strlen(src);
9      if (size == 0)
10         return (slen);
11     i = 0;
12     while (src[i] && i < size - 1)
13     {
14         dest[i] = src[i];
15         i++;
16     }
17     dest[i] = '\0';
18     return (slen);
19 }
```

The `size == 0` test comes first because `size - 1` on an unsigned 0 would be SIZE_MAX (unlimited copy), and because with size 0 there is no room even for `'\0'` — dest is not touched at all.

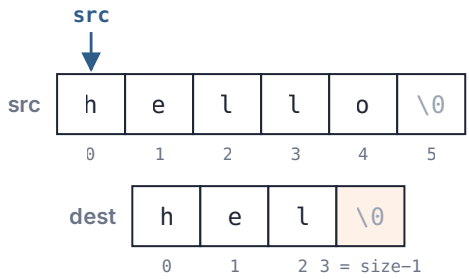


Figure — ft_strlcpy(dest, "hello", 4) copies 3 characters + '\0' and returns 5

TRACE — SIZE = 4

I	SRC[I]	I < SIZE−1 (3)?	ACTION
0	'h'	yes	dest[0] = 'h'
1	'e'	yes	dest[1] = 'e'
2	'l'	yes	dest[2] = 'l'
3	'l'	no	stop → dest[3] = '\0'; return 5

ft_strlcat

ft_strlcat.c · uses `ft_strlen`

STRING | NO ALLOCATION | MIRRORS LIBC STRLCAT (BSD)

```
size_t        ft_strlcat(char *dest, const char *src, size_t size);
```

Appends `src` to the end of `dest`, where `size` is the **full size of the dest buffer** (not the space left). It writes at most `size − strlen(dest) − 1` characters and terminates. It returns the length of the string it tried to create: `initial strlen(dest) + strlen(src)`.

> **KEY IDEA**
>
> The special case: if `dest` has no `'\0'` within its first `size` bytes (i.e. `size ≤ strlen(dest)`), the buffer is considered already full: nothing is written and the function returns **`size + strlen(src)`**. This is why the length of dest is measured with a loop bounded by `size`, not with strlen.

FT_STRLCAT.C · AS WRITTEN IN THE REPOSITORY

```
1  size_t  ft_strlcat(char *dest, const char *src, size_t size)
2  {
3      size_t  dlen;
4      size_t  slen;
5      size_t  i;
6      char    *dp;
7
8      dlen = 0;
9      slen = ft_strlen(src);
10     while (dlen < size && dest[dlen])
11         dlen++;
12     if (size == 0)
13         return (slen);
14     if (dlen == size)
15         return (size + slen);
16     i = 0;
```

```
17      dp = dest + dlen;
18      while (src[i] && i < size - dlen - 1)
19          *dp++ = src[i++];
20      *dp = '\0';
21      return (dlen + slen);
22  }
```

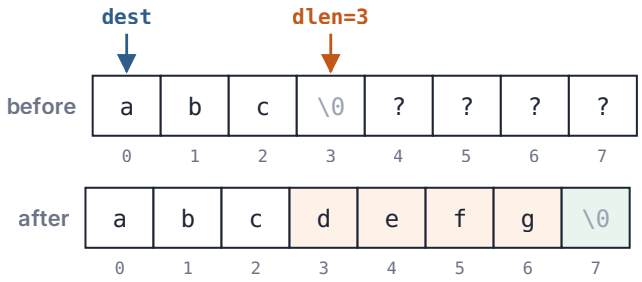


Figure — ft_strlcat(dest, "defghijkl", 8): room for 8−3−1 = 4 characters; returns 3 + 9 = 12

CASE	DLEN	WRITTEN	RETURN
dest "abc", size 8, src "defghijkl"	3	"defg" + '\0' → "abcdefg"	3 + 9 = 12
dest "abc", size 3	3 (= size)	nothing	size + slen = 3 + 9 = 12
dest "abc", size 2	2 (loop stops)	nothing	2 + 9 = 11
size 0	0	nothing	slen = 9
dest "abc", size 20, src "de"	3	"de" → "abcde"	5

Order of the checks matters: `dlen` is computed first; then `size == 0` returns `slen` (which also equals `size + slen`, consistent with the rule); then `dlen == size`. Only after these is `size - dlen - 1` safe (≥ 0).

> **PITFALL**
> The loop bounded by `size` means `ft_strlcat(NULL, "abc", 0)` does not crash (dest is never read), but `slen` is computed first so `src` may never be NULL. This matches the BSD implementation.

ft_strchr

ft_strchr.c

STRING NO ALLOCATION MIRRORS LIBC STRCHR

```
char        *ft_strchr(const char *s, int c);
```

Returns a pointer to the **first** occurrence of `c` in `s`, or NULL. The terminator is considered part of the string: `ft_strchr(s, '\0')` returns a pointer to the end of `s` — a common trick to find the end of a string.

FT_STRCHR.C · AS WRITTEN IN THE REPOSITORY

```
1  char    *ft_strchr(const char *s, int c)
```

```c
{
    char    *ptr;

    ptr = (char *)s;
    while (*ptr)
    {
        if (*ptr == (unsigned char)c)
            return (ptr);
        ptr++;
    }
    if ((unsigned char)c == '\0')
        return (ptr);
    return (NULL);
}
```



Figure — ft_strchr("banana", 'a') → s + 1

IN THIS REPO

`*ptr` is a `char` (signed on x86: −128…127) and `(unsigned char)c` is 0…255. For any byte ≥ 128 they can never be equal. Test: `ft_strchr("café", 0xE9)` returns NULL, libc returns the address. The standard says `c` is converted to **char**.

CORRECTED FT_STRCHR

```c
char    *ft_strchr(const char *s, int c)
{
    while (*s)
    {
        if (*s == (char)c)
            return ((char *)s);
        s++;
    }
    if ((char)c == '\0')
        return ((char *)s);
    return (NULL);
}
```

ft_strrchr

ft_strrchr.c

STRING NO ALLOCATION MIRRORS LIBC STRRCHR

```c
char        *ft_strrchr(char const *s, int c);
```

Like strchr but returns the **last** occurrence. Strategy here: walk the whole string forward and remember the latest match in `ptr` (initially NULL). An alternative is to go to the end and walk backward; both are O(n).

FT_STRRCHR.C · AS WRITTEN IN THE REPOSITORY

```c
char    *ft_strrchr(char const *s, int c)
{
    const char  *ptr;
    size_t      i;

    ptr = NULL;
    i = 0;
    while (s[i])
    {
        if (s[i] == (unsigned char)c)
            ptr = &s[i];
        i++;
    }
    if ((unsigned char)c == '\0')
        ptr = &s[i];
    return ((char *)ptr);
}
```

TRACE — FT_STRRCHR("BANANA", 'A')

I	S[I]	MATCH 'A'?	PTR
0	'b'	no	NULL
1	'a'	yes	&s[1]
2	'n'	no	&s[1]
3	'a'	yes	&s[3]
4	'n'	no	&s[3]
5	'a'	yes	&s[5]
6	'\0'	loop ends	return &s[5]

IN THIS REPO

Same signed/unsigned comparison bug as ft_strchr. Fix: `if (s[i] == (char)c)` and `if ((char)c == '\0')`.

ft_strncmp

ft_strncmp.c

STRING NO ALLOCATION MIRRORS LIBC STRNCMP

```c
int         ft_strncmp(const char *s1, const char *s2, size_t n);
```

Compares at most `n` characters, stopping early at the first difference or at the end of either string. Returns < 0, 0 or > 0, comparing as `unsigned char`. Typical use: `ft_strncmp(line, "exit", 5) == 0` (5 includes the terminator → exact match).

FT_STRNCMP.C · AS WRITTEN IN THE REPOSITORY

```c
int ft_strncmp(const char *s1, const char *s2, size_t n)
{
    size_t              i;
    const unsigned char *p1;
    const unsigned char *p2;
```

```
 6
 7      if (n == 0)
 8          return (0);
 9      i = 0;
10      p1 = (const unsigned char *)s1;
11      p2 = (const unsigned char *)s2;
12      while (i < n && *p1 && *p2 && (*p1 == *p2))
13      {
14          p1++;
15          p2++;
16          i++;
17      }
18      return (*p1 - *p2);
19  }
```

Walk-through of the loop condition: keep going while (1) fewer than n characters compared, (2) neither string has ended, (3) the characters are equal. When it stops, `*p1 - *p2` is the answer. If it stopped because `i == n`, the pointers are on position n — that is fine here only because...

PITFALL

...it is **not** always fine. With `ft_strncmp("abcX", "abcY", 3)`, the loop stops at i = 3 and returns `'X' - 'Y'` = −1 instead of 0! Let us check the code again: the loop exits with `i == n`, and the function returns `*p1 - *p2` without testing `i`. Worse, the very common test `ft_strncmp("ab", "abc", 2)` returns −99 (`'\0' - 'c'`) where libc returns 0. Verified by running both.

TRACE — FT_STRNCMP("ABCX", "ABCY", 3)

I	*P1	*P2	CONTINUE?
0	'a'	'a'	yes
1	'b'	'b'	yes
2	'c'	'c'	yes
3	'X'	'Y'	i < n fails → exit
			return 'X' − 'Y' = −1 ✗ (libc: 0)

The fix is the same guard ft_memcmp already has: `if (i == n) return (0);` before the final subtraction. (This bug is listed in Chapter 11 and was confirmed by running the code.)

ft_strnstr

ft_strnstr.c

STRING NO ALLOCATION MIRRORS LIBC STRNSTR (BSD)

```
char        *ft_strnstr(const char *haystack, const char *needle, size_t
len);
```

Finds the first occurrence of `needle` in `haystack`, but searches only the first `len` characters: the whole match must fit inside them. Empty needle → return haystack. Not found → NULL.

FT_STRNSTR.C · AS WRITTEN IN THE REPOSITORY

```
1  char    *ft_strnstr(const char *haystack, const char *needle, size_t len)
2  {
```

```c
    const char  *big;
    const char  *little;
    size_t      i;
    size_t      j;

    if (!*needle)
        return ((char *)haystack);
    j = 0;
    while (j < len && *haystack)
    {
        i = 0;
        big = haystack;
        little = needle;
        while (j + i < len && *big && *little && (*big == *little))
        {
            big++;
            little++;
            i++;
        }
        if (*little == '\0')
            return ((char *)haystack);
        j++;
        haystack++;
    }
    return (NULL);
}
```

Two nested loops. The outer one moves a **start position** `j` along the haystack. The inner one compares needle against haystack from that start, with `j + i < len` keeping every compared character inside the window. If the inner loop consumed the entire needle (`*little == '\0'`), we have a match at the current start. Complexity O(len × |needle|).

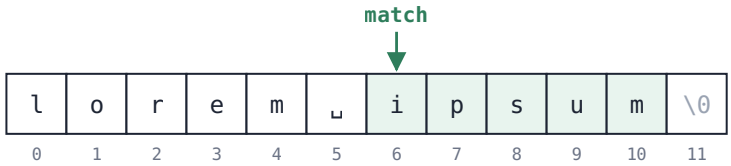


Figure — ft_strnstr("lorem ipsum", "ipsum", 11) → haystack + 6. With len = 10 the 'm' (index 10) is outside → NULL

TRACE — NEEDLE "IPSUM", LEN = 11

J (START)	INNER LOOP	RESULT
0 'l'	'l' ≠ 'i'	next start
1–5	first char differs	next start
6 'i'	i,p,s,u,m all equal; needle ends	return haystack + 6

ft_strdup

ft_strdup.c · uses `ft_strlen`, `ft_strlcpy`

ALLOCATING STRING · ALLOCATES (MALLOC) · MIRRORS LIBC STRDUP

```c
char        *ft_strdup(const char *s);
```

Allocates `strlen(s) + 1` bytes on the heap and copies `s` into it. The returned string belongs to the caller, who must free it.

FT_STRDUP.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"
#include <stdio.h>
#include <stdlib.h>

char    *ft_strdup(const char *s)
{
    size_t  slen;
    char    *dup;

    if (!s)
        return (NULL);
    slen = ft_strlen(s);
    dup = malloc(slen + 1);
    if (!dup)
        return (NULL);
    ft_strlcpy(dup, s, slen + 1);
    return (dup);
}

int main(void)
{
    printf("%s\n", ft_strdup(NULL));
    return (0);
}
```

IN THIS REPO

This file still contains a test `main()` that calls `printf` (forbidden function) and passes NULL to printf's `%s`. Compiled into `libft.a`, the archive now defines `main`: any program that links the library and also uses ft_strdup gets “multiple definition of main”. Norm error too. Delete lines 20–24 and the `<stdio.h>` include.

The `if (!s) return (NULL);` is an extra protection; libc's strdup would crash on NULL. Using `ft_strlcpy(dup, s, slen + 1)` copies all slen characters and the terminator.

ft_atoi

ft_atoi.c · uses `ft_isdigit`

CONVERSION · NO ALLOCATION · MIRRORS LIBC ATOI

```c
int         ft_atoi(const char *str);
```

Converts the beginning of a string to an int, following the libc rules:

1. skip leading whitespace: space and `\t \n \v \f \r` (ASCII 9–13);
2. accept **one** optional sign, `+` or `-` (so `"--5"` and `"+-5"` give 0);
3. read digits until the first non-digit; ignore the rest.

FT_ATOI.C · AS WRITTEN IN THE REPOSITORY

```c
#include <limits.h>

static int  issign(unsigned char c)
```

```c
{
    if (c == '-' || c == '+')
        return (1);
    return (0);
}

static int  ft_isspace(unsigned char c)
{
    if ((c >= 9 && c <= 13) || c == ' ')
        return (1);
    return (0);
}

static int  ft_overflow(int sign)
{
    if (sign == -1)
        return (0);
    return (-1);
}

int ft_atoi(const char *str)
{
    long long   result;
    int         sign;

    result = 0;
    sign = 1;
    while (*str && ft_isspace(*str))
        str++;
    if (issign(*str))
    {
        if (*str == '-')
            sign = -1;
        str++;
    }
    while (*str && ft_isdigit(*str))
    {
        if (result > (9223372036854775807 - (*str - '0')) / 10)
            return (ft_overflow(sign));
        result = result * 10 + (*str - '0');
        str++;
    }
    return ((int)(result * sign));
}
```

DIGIT TO VALUE

`*str - '0'` : digit characters are consecutive in ASCII, so `'7' - '0'` = 55 − 48 = 7. Building the number: `result = result × 10 + digit` shifts the previous digits one decimal place left.

TRACE — FT_ATOI(" \T-42ABC")

STR POINTS TO	PHASE	SIGN	RESULT
' '	skip space	1	0
'\t'	skip space	1	0
'-'	sign	−1	0
'4'	digit	−1	0×10 + 4 = 4
'2'	digit	−1	4×10 + 2 = 42
'a'	not a digit → stop	−1	return (int)(42 × −1) = −42

THE OVERFLOW LOGIC IN THIS REPO

Most students just accumulate in an int. This version accumulates in `long long` and, *before* each step, checks whether `result × 10 + digit` would exceed LLONG_MAX (9 223 372 036 854 775 807) by testing `result > (LLONG_MAX − digit) / 10`. On overflow it returns −1 for positive and 0 for negative numbers. Why those values? Real atoi is `(int)strtol(str, NULL, 10)`; on overflow strtol returns LONG_MAX or LONG_MIN, and casting those to int keeps the low 32 bits: `(int)LONG_MAX` = −1 and `(int)LONG_MIN` = 0. This reproduces glibc exactly.

INPUT	FT_ATOI	LIBC ATOI	WHY
"2147483647"	2147483647	2147483647	INT_MAX fits
"-2147483648"	−2147483648	−2147483648	INT_MIN: 2147483648 fits in long long, times −1, cast to int
"2147483648"	−2147483648	−2147483648	fits in long long; cast to int wraps (implementation-defined)
"9999999999999999999"	−1	−1	exceeds long long → overflow branch
"-9999999999999999999"	0	0	negative overflow branch
"--1"	0	0	second sign is not a digit

IN THIS REPO

The file includes `<limits.h>` (unused — the constant is written literally) but not `"libft.h"`, so `ft_isdigit` is implicitly declared → build fails.

QUESTIONS YOU MAY BE ASKED

Q What does strlcpy return and why is that useful?
→ strlen(src). If the return is ≥ size, the copy was truncated; the caller knows how big the buffer needed to be (return + 1).

Q What does strlcat return when size ≤ strlen(dest)?
→ size + strlen(src); nothing is appended.

Q What does ft_strchr(s, '\0') return?
→ A pointer to the terminating '\0' of s, not NULL.

Q Why does strchr take const char * but return char *?
→ So it works on both const and non-const strings, mirroring libc; it casts away const, and the caller is responsible for not writing through it if the original was const.

Q Why compare as unsigned char in strncmp?
→ The standard requires it: bytes ≥ 128 must compare greater than ASCII bytes.

Q What does ft_strnstr return when needle is ""?
→ haystack itself.

Q What does ft_atoi(" +-12") return?
→ 0: after '+', the next char '-' is not a digit.

Q Why does atoi skip \v and \f?
→ isspace in the C locale is exactly ' ', \t, \n, \v, \f, \r (9–13 and 32).

Q Where is the memory returned by ft_strdup freed?
→ Not by ft_strdup — the caller must call free on it.

CHAPTER 7

Part 2: New Strings & Callbacks

> *Part 2 functions build new strings on the heap. Every one of them follows the same discipline: compute the exact size, malloc size + 1, check for NULL, fill, terminate. ft_split is the hardest function in the project — it allocates an array of allocated strings and must clean up after a failure.*

KEY IDEA

The allocating-function checklist — apply it to every function in this chapter and the evaluator will have nothing to say:

1. Protect against NULL inputs if your design does (this repo returns NULL).
2. Compute the exact length needed.
3. `malloc(len + 1)` — the +1 is for `'\0'`.
4. `if (!ptr) return (NULL);`
5. Fill, then write the terminator.
6. If several allocations happen and one fails, free the earlier ones.

ft_substr

ft_substr.c · uses `ft_strlen`

ALLOCATING STRING · ALLOCATES (MALLOC)

```
char        *ft_substr(char const *s, unsigned int start, size_t len);
```

PARAMETER / RETURN	TYPE	MEANING
`s`	`char const *`	the source string
`start`	`unsigned int`	index where the substring begins
`len`	`size_t`	maximum length of the substring
`return`	`char *`	new string, or NULL if malloc fails

Returns a new string with up to `len` characters of `s` starting at `start`. The difficulty is the edge cases, handled by the static helper `guard`, which computes how many characters to really copy:

FT_SUBSTR.C · AS WRITTEN IN THE REPOSITORY

```
1  #include "libft.h"
2
3  static size_t   guard(size_t len, size_t slen, unsigned int start)
4  {
5      size_t  count;
6
7      if (start >= slen)
```

```c
        count = 0;
    else if (len > slen - start)
        count = slen - start;
    else
        count = len;
    return (count);
}

char    *ft_substr(char const *s, unsigned int start, size_t len)
{
    char    *sub;
    size_t  i;
    size_t  count;
    size_t  slen;

    if (!s)
        return (NULL);
    slen = ft_strlen(s);
    count = guard(len, slen, start);
    sub = malloc(count + 1);
    if (!sub)
        return (NULL);
    i = 0;
    while (i < count)
    {
        sub[i] = s[start + i];
        i++;
    }
    sub[i] = '\0';
    return (sub);
}
```

CASE	COUNT	EXAMPLE → RESULT
`start ≥ slen`	0	substr("hello", 10, 3) → "" (an allocated empty string, not NULL)
`len > slen − start`	slen − start	substr("hello", 1, 100) → "ello" (allocates 5 bytes, not 101)
otherwise	len	substr("hello", 1, 2) → "el"

PITFALL

Why not just `malloc(len + 1)`? With `len = SIZE_MAX` that is `malloc(0)` (wrap-around), and with `len = 1000000` on a 5-char string it wastes memory. Testers check both. Also note `len > slen − start` is written so it never computes `start + len`, which could overflow.

ft_strjoin

ft_strjoin.c · uses `ft_strlen`

ALLOCATING STRING · ALLOCATES (MALLOC)

```c
char        *ft_strjoin(char const *s1, char const *s2);
```

Allocates `len1 + len2 + 1` bytes and writes s1 followed by s2. Neither input is modified or freed.

FT_STRJOIN.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"
#include <stdint.h>

char    *ft_strjoin(char const *s1, char const *s2)
{
    size_t  s1len;
    size_t  s2len;
    char    *joined;
    char    *ptr;

    if (!s1 || !s2)
        return (NULL);
    s1len = ft_strlen(s1);
    s2len = ft_strlen(s2);
    if (s2len == SIZE_MAX || s1len > SIZE_MAX - s2len - 1)
        return (NULL);
    joined = malloc(s1len + s2len + 1);
    if (!joined)
        return (NULL);
    ptr = joined;
    while (*s1)
        *ptr++ = *s1++;
    while (*s2)
        *ptr++ = *s2++;
    *ptr = '\0';
    return (joined);
}
```

Line 15 is an overflow guard: if `s1len + s2len + 1` would exceed SIZE_MAX it returns NULL instead of allocating a wrapped tiny size. In practice no string can be that long, but the evaluator will appreciate the reasoning. The copy uses a separate cursor `ptr` so that `joined` still points to the start when it is returned.

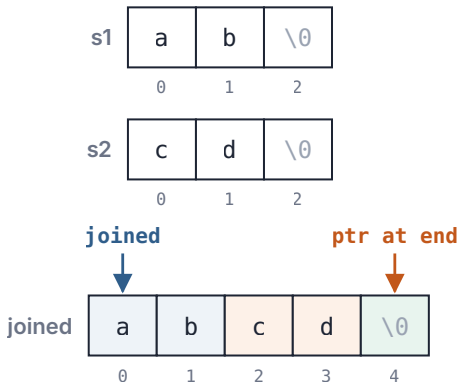


Figure — ft_strjoin("ab", "cd")

IN THIS REPO

`ft_strjoin` has no prototype in `libft.h`. The file compiles (it includes the header and defines the function) but every *caller* fails with an implicit-declaration error.

ft_strtrim

ft_strtrim.c · uses `ft_strlen`, `ft_strdup`, `ft_strchr`, `ft_substr`

ALLOCATING STRING · ALLOCATES (MALLOC)

```
char        *ft_strtrim(char const *s1, char const *set);
```

Removes every character that belongs to `set` from the **beginning and the end** of s1 (not the middle). `set` is not a word to remove but a *set of characters*: `ft_strtrim("xxhixyx", "xy")` → `"hi"`.

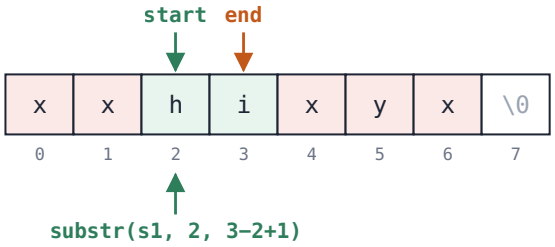


Figure — start skips leading set chars, end skips trailing ones, then one ft_substr call

FT_STRTRIM.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"

char    *ft_strtrim(char const *s1, char const *set)
{
    size_t  start;
    size_t  end;

    if (!s1 || !set)
        return (NULL);
    if (ft_strlen(s1) == 0)
        return (ft_strdup(s1));
    start = 0;
    while (s1[start] && ft_strchr(set, s1[start]))
        start++;
    if (s1[start] == '\0')
        return (ft_strdup(""));
    end = ft_strlen(s1) - 1;
    while (end > start && ft_strchr(set, s1[end]))
        end--;
    return (ft_substr(s1, start, end - start + 1));
}
```

Membership test: `ft_strchr(set, ch)` is non-NULL when `ch` is in set. But careful — strchr also matches `'\0'`! That is why the first loop checks `s1[start]` first: it stops at the end of s1 before ever asking strchr about the terminator. If everything was trimmed (`s1[start] == '\0'`), it returns an allocated `""`. The second loop is bounded by `end > start`, so it cannot cross the first.

TRACE — FT_STRTRIM("XXHIXYX", "XY")

STEP	START	END	NOTE
skip front	0 'x' ∈ set		
	1 'x' ∈ set		
	2 'h' ∉ set → stop		
init end	2	6 (len − 1)	
skip back	2	6 'x' ∈ → 5	
	2	5 'y' ∈ → 4	
	2	4 'x' ∈ → 3	
	2	3 'i' ∉ → stop	
return			ft_substr(s1, 2, 2) → "hi"

ft_split

ft_split.c · uses `ft_substr`

ALLOCATING STRING · ALLOCATES (MALLOC)

```
char        **ft_split(char const *s, char c);
```

PARAMETER / RETURN	TYPE	MEANING
`s`	`char const *`	string to split
`c`	`char`	the delimiter character
`return`	`char **`	NULL-terminated array of new strings, or NULL on any malloc failure

`char **` is a pointer to the first element of an array whose elements are `char *`. The array ends with a NULL pointer so the caller can loop `while (arr[i])` without knowing the count. Consecutive delimiters do **not** create empty words.

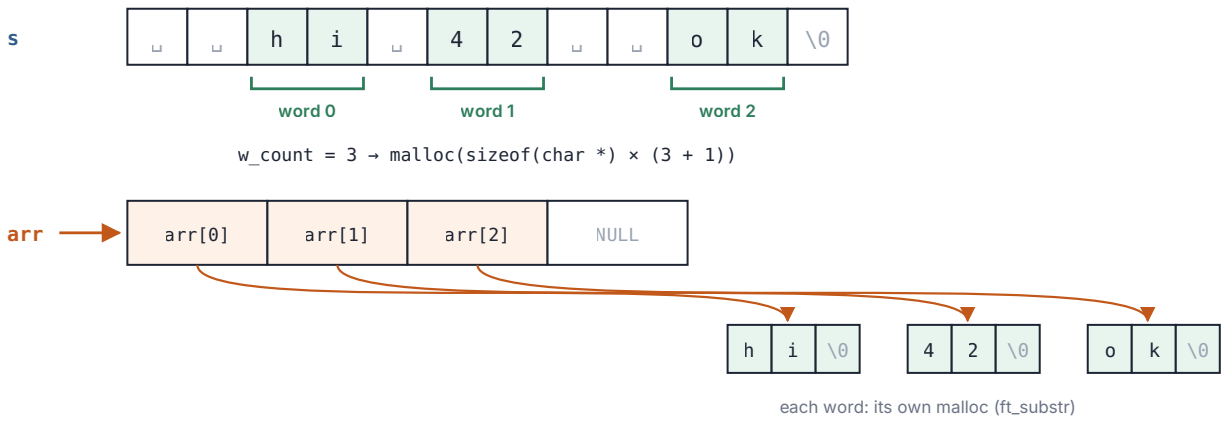


Figure — ft_split(" hi 42 ok", ' '): 1 allocation for the array + 1 per word = 4 mallocs

FT_SPLIT.C · AS WRITTEN IN THE REPOSITORY

```
1  static size_t   w_count(const char *s, char c)
```

```
2  {
3      int     found;
4      size_t  count;
5
6      count = 0;
7      found = 0;
8      while (*s)
9      {
10         if (*s == c)
11             found = 0;
12         else if (found == 0)
13         {
14             found = 1;
15             count++;
16         }
17         s++;
18     }
19     return (count);
20 }
21
22 static void free_arr(char **arr, size_t n)
23 {
24     size_t  i;
25
26     i = 0;
27     while (i < n)
28     {
29         free(arr[i]);
30         i++;
31     }
32     free(arr);
33 }
34
35 static size_t   w_len(char const *s, char c)
36 {
37     size_t  len;
38
39     len = 0;
40     while (*s && *s != c)
41     {
42         len++;
43         s++;
44     }
45     return (len);
46 }
47
48 char    **ft_split(char const *s, char c)
49 {
50     char    **arr;
51     size_t  i;
52
53     if (!s)
54         return (NULL);
55     arr = malloc(sizeof(char *) * (w_count(s, c) + 1));
56     if (!arr)
57         return (NULL);
58     i = 0;
59     while (*s)
60     {
61         if (*s != c)
62         {
63             arr[i] = ft_substr(s, 0, w_len(s, c));
64             if (!arr[i])
```

```
65                  return (free_arr(arr, i), NULL);
66              i++;
67              s += w_len(s, c);
68          }
69          else
70              s++;
71      }
72      arr[i] = NULL;
73      return (arr);
74  }
```

W_COUNT — COUNTING WORDS WITH A STATE FLAG

`found` remembers whether we are currently inside a word. A word starts at a non-delimiter character when `found` was 0; that is the only moment count increases. A delimiter resets `found` to 0. This is a tiny **finite-state machine** with two states.

TRACE — W_COUNT(" HI 42 OK", ' ')

*S	IS C?	FOUND BEFORE	ACTION	COUNT
'␣'	yes	0	found = 0	0
'␣'	yes	0	found = 0	0
'h'	no	0	found = 1, count++	1
'i'	no	1	–	1
'␣'	yes	1	found = 0	1
'4'	no	0	found = 1, count++	2
'2'	no	1	–	2
'␣'	yes	1	found = 0	2
'␣'	yes	0	found = 0	2
'o'	no	0	found = 1, count++	3
'k'	no	1	–	3

THE MAIN LOOP

Walk `s`. On a delimiter, skip it. On a word start, measure the word with `w_len` (characters until the next delimiter or end), copy it with `ft_substr(s, 0, len)`, then jump `s` past the word. Finally terminate the array with NULL.

CLEANING UP ON FAILURE

If the *k*-th ft_substr fails, words 0…k–1 are already allocated. `free_arr(arr, i)` frees each of them and then the array itself. `return (free_arr(arr, i), NULL);` uses the **comma operator**: evaluate the left side (the call), discard its value, and the whole expression has the value of the right side (NULL). It fits the Norm's 25-line limit.

INPUT	RESULT
`ft_split("", ' ')`	`{NULL}` — a 1-element array, not NULL
`ft_split(" ", ' ')`	`{NULL}`
`ft_split("abc", '\0')`	`{"abc", NULL}`: no delimiter is ever met
`ft_split(NULL, ' ')`	NULL

TIP

How to free the result (the caller's job):

```
i = 0;
while (arr[i])
    free(arr[i++]);   /* each word */
free(arr);            /* then the array */
```

ft_itoa

ft_itoa.c

ALLOCATING CONVERSION ALLOCATES (MALLOC)

```
char        *ft_itoa(int n);
```

The inverse of atoi: returns a new string representing `n` in base 10. Plan: count the characters (digits + 1 for a minus sign), allocate, then write digits **from right to left** using `% 10` (last digit) and `/ 10` (drop last digit).

FT_ITOA.C · AS WRITTEN IN THE REPOSITORY

```
 1  #include <stdlib.h>
 2
 3  static size_t   ft_intlen(long nbr)
 4  {
 5      size_t  len;
 6
 7      len = 0;
 8      if (nbr == 0)
 9          return (1);
10      if (nbr < 0)
11      {
12          len++;
13          nbr = -nbr;
14      }
15      while (nbr > 0)
16      {
17          len++;
18          nbr /= 10;
19      }
20      return (len);
21  }
22
23  char    *ft_itoa(int n)
24  {
25      char    *ptr;
26      size_t  nlen;
27      long    nbr;
28
29      nbr = n;
30      nlen = ft_intlen(nbr);
31      ptr = malloc(nlen + 1);
32      if (!ptr)
33          return (NULL);
34      ptr[nlen] = '\0';
```

```
35      if (nbr < 0)
36      {
37          ptr[0] = '-';
38          nbr = -nbr;
39      }
40      if (nbr == 0)
41          ptr[0] = '0';
42      while (nbr > 0)
43      {
44          ptr[--nlen] = nbr % 10 + '0';
45          nbr /= 10;
46      }
47      return (ptr);
48  }
```

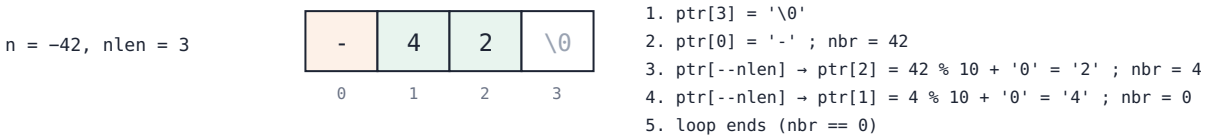


Figure — Filling the buffer from the end for n = −42

Why copy `n` into `long nbr`? Because `-n` for `n = INT_MIN` overflows an int; in a long, −(−2 147 483 648) = 2 147 483 648 fits. Why `ft_intlen` handles 0 separately: the `while (nbr > 0)` loop would count 0 digits for zero. For `--nlen`: pre-decrement — decrease first, then use as index, so the first digit goes to `ptr[nlen − 1]`, just before the terminator.

N	FT_INTLEN	MALLOC	RESULT
0	1	2	"0"
−42	3	4	"-42"
2147483647	10	11	"2147483647"
−2147483648	11	12	"-2147483648"

ft_strmapi

ft_strmapi.c · uses `ft_strlen`

HIGHER-ORDER STRING · ALLOCATES (MALLOC)

```
char        *ft_strmapi(char const *s, char (*f)(unsigned int, char));
```

Creates a **new** string where each character is the result of `f(index, original_char)`. The original is untouched (it is const). “map” is the functional-programming name for “apply a function to each element and collect the results”.

FT_STRMAPI.C · AS WRITTEN IN THE REPOSITORY

```
1  #include "libft.h"
2
3  char    *ft_strmapi(char const *s, char (*f)(unsigned int, char))
4  {
5      char    *str;
6      size_t  i;
```

```
    size_t  slen;

    if (!s || !f)
        return (NULL);
    slen = ft_strlen(s);
    str = malloc(slen + 1);
    if (!str)
        return (NULL);
    i = 0;
    while (i < slen)
    {
        str[i] = f(i, s[i]);
        i++;
    }
    str[i] = '\0';
    return (str);
}
```

EXAMPLE CALLBACK

```
char    shift_by_index(unsigned int i, char c)
{
    return (c + i);
}
/* ft_strmapi("aaaa", shift_by_index)  →  "abcd"  (new string) */
```

Small type detail: the loop index is `size_t i` but `f` wants an `unsigned int`; the value is converted implicitly. Strings longer than UINT_MAX are not a realistic concern.

ft_striteri

ft_striteri.c

HIGHER-ORDER STRING · NO ALLOCATION

```
void        ft_striteri(char *s, void (*f)(unsigned int, char *));
```

Applies `f` to each character **in place**. It passes the *address* `&s[i]` so `f` can modify the character. Nothing is allocated, nothing is returned. Compare with strmapi:

	FT_STRMAPI	FT_STRITERI
callback	`char f(unsigned int, char)`	`void f(unsigned int, char *)`
receives	a copy of the character	the address of the character
result	new malloc'd string	modifies s itself
s is	`char const *`	`char *` (must be writable)

FT_STRITERI.C · AS WRITTEN IN THE REPOSITORY

```
#include "libft.h"

void    ft_striteri(char *s, void (*f)(unsigned int, char *))
{
    unsigned int    i;

```

```
    if (!s || !f)
        return ;
    i = 0;
    while (s[i])
    {
        f(i, &s[i]);
        i++;
    }
}
```

EXAMPLE CALLBACK

```
void    upper_even(unsigned int i, char *c)
{
    if (i % 2 == 0)
        *c = ft_toupper(*c);     /* write through the pointer */
}
/* char s[] = "hello";  ft_striteri(s, upper_even);  →  s is now "HeLlO" */
/* ft_striteri("hello", upper_even) would crash: string literals are read-only */
```

QUESTIONS YOU MAY BE ASKED

Q What does ft_substr("hello", 7, 2) return?
→ An allocated empty string "" (1 byte), not NULL — so the caller can always free the result.

Q Why does your ft_split return {NULL} for an empty string instead of NULL?
→ NULL means allocation failure. An empty result is a valid array with zero words.

Q How many mallocs does ft_split("a b c", ' ') do?
→ 4: one for the array of 4 pointers (3 words + NULL), one per word.

Q What happens if malloc fails on the third word of split?
→ free_arr frees the two words already allocated and the array, then NULL is returned — no leak.

Q Why use long in ft_itoa?
→ To negate INT_MIN without signed overflow.

Q Difference between strmapi and striteri?
→ strmapi builds a new string from f's return values; striteri passes each character's address to f, which modifies the string in place.

Q Does ft_strtrim remove characters in the middle?
→ No, only from both ends. "a b" trimmed with " " stays "a b".

Q What is the comma operator in return (free_arr(arr, i), NULL);?
→ It evaluates the left operand for its side effect, then yields the right operand as the value of the expression.

CHAPTER 8

Output Functions

> *Four functions that write to a file descriptor using only write(2). ft_putnbr_fd is the interesting one: recursion prints digits in the right order without any buffer.*

All four have the same shape: `void f(…, int fd)`. They return nothing and do not report write errors. Passing `fd = 1` prints to the terminal, `2` to stderr, or a value returned by `open()` to write into a file.

ft_putchar_fd

ft_putchar_fd.c

OUTPUT NO ALLOCATION

```c
void        ft_putchar_fd(char c, int fd);
```

FT_PUTCHAR_FD.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"
// #include <fcntl.h>
#include <unistd.h>

void    ft_putchar_fd(char c, int fd)
{
    write(fd, &c, 1);
}

/*
int main(void)
{
    int fd;

    fd = open("text.test", O_CREAT | O_WRONLY);
    if (fd == -1)
        return (-1);
    ft_putchar_fd('a', fd);
    return (0);
}
*/
```

`write` wants the **address** of bytes, not a value, so we pass `&c`: the address of the parameter `c` (a local copy on the stack), and a count of 1. The commented-out `main` at the bottom is ignored by the compiler; comments outside functions are allowed by the Norm, but evaluators often ask you to remove leftover test code.

ft_putstr_fd

ft_putstr_fd.c · uses `ft_strlen`

OUTPUT NO ALLOCATION

```
void        ft_putstr_fd(char *s, int fd);
```

FT_PUTSTR_FD.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"
#include <fcntl.h>

void    ft_putstr_fd(char *s, int fd)
{
    if (!s)
        return ;
    write(fd, s, ft_strlen(s));
}
/*
int main(void)
{
    int fd;

    fd = open("text.test", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1)
        return (-1);
    ft_putstr_fd("Hamza is testing all the time", fd);
    return (0);
}
*/
```

One single `write` of the whole string (more efficient than one write per character: each write is a system call). The NULL check avoids a crash in ft_strlen.

IN THIS REPO

`write` is declared in `<unistd.h>`, but this file includes `<fcntl.h>` (for `open`, used only in the commented test). Result: implicit declaration of `write` → compile error. ft_putchar_fd does include unistd.h, which is why it compiles.

ft_putendl_fd

ft_putendl_fd.c · uses `ft_putstr_fd`, `ft_putchar_fd`

OUTPUT NO ALLOCATION

```
void        ft_putendl_fd(char *s, int fd);
```

FT_PUTENDL_FD.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"
#include <fcntl.h>

```

```c
void    ft_putendl_fd(char *s, int fd)
{
    if (!s)
        return ;
    ft_putstr_fd(s, fd);
    ft_putchar_fd('\n', fd);
}
/*
int main(void)
{
    // int  fd;
    // fd = open("text.test", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    // if (fd == -1)
    //  return (-1);
    ft_putendl_fd("Hamza is testing all the time", 1);
    return (0);
}
*/
```

The string followed by a newline (“end line”). With a NULL `s` this version prints nothing at all (not even the newline) — a choice you should be able to justify.

ft_putnbr_fd

ft_putnbr_fd.c · uses `ft_putstr_fd`, `ft_putchar_fd`

OUTPUT NO ALLOCATION

```c
void        ft_putnbr_fd(int n, int fd);
```

Prints an int in decimal, without malloc and without itoa. The problem: digits are easy to get from the right (`n % 10`) but must be printed from the left. Recursion solves it: first print everything except the last digit (`n / 10`), then print the last digit.

FT_PUTNBR_FD.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"

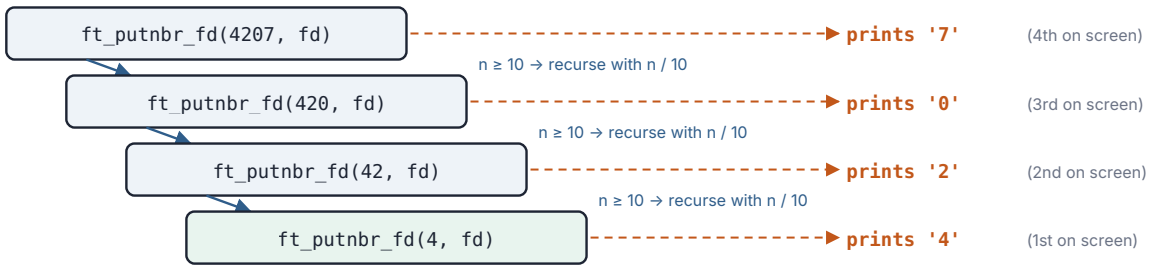
void    ft_putnbr_fd(int n, int fd)
{
    char    c;

    if (n == -2147483648)
    {
        ft_putstr_fd("-2147483648", fd);
        return ;
    }
    if (n < 0)
    {
        ft_putchar_fd('-', fd);
        n = -n;
    }
    if (n >= 10)
        ft_putnbr_fd(n / 10, fd);
    c = n % 10 + '0';
    ft_putchar_fd(c, fd);
}

/*
```

```c
int main(void)
{
    int n;

    n = -250;
    ft_putnbr_fd(n, 1);
    return (0);
}
*/
```



Calls go down (most-significant digit reached last); prints happen on the way back up, so '4' is printed first.

Figure — The call stack of ft_putnbr_fd(4207, fd)

INT_MIN is handled first because `n = -n` would overflow. **Negative numbers**: print `'-'`, make n positive, continue. **Base case**: when n < 10 there is no recursive call; a single digit is printed. Maximum recursion depth: 10.

TRACE — FT_PUTNBR_FD(−250, 1) PRINTS -250

FRAME	N	N ≥ 10?	AFTER THE RECURSIVE CALL RETURNS
1	−250 → '-' printed, n = 250	yes → call(25)	print '0'
2	25	yes → call(2)	print '5'
3	2	no	print '2' ← first digit on screen

QUESTIONS YOU MAY BE ASKED

Q Why pass &c and not c to write?
→ write takes a pointer to a buffer of bytes; &c is the address of the one-byte buffer holding the character.

Q Why does putnbr need a special case for -2147483648?
→ Its positive counterpart 2147483648 does not fit in an int, so n = -n would overflow.

Q How would you print INT_MIN without the special case?
→ Use a long: long nb = n; then work with nb, whose negation fits.

Q Why is one write of the whole string better than a loop of putchar?
→ Each write is a system call (a switch into the kernel); fewer calls means much faster output.

Q What happens if fd is invalid (e.g. -1)?
→ write returns -1 and sets errno; nothing is printed and our functions ignore it.

CHAPTER 9

The t_list Struct & Linked Lists

> *A linked list is a chain of heap-allocated nodes, each holding a pointer to its data and a pointer to the next node. This chapter explains how the struct works in memory, why some functions take t_list ** and others t_list *, and traces all nine list functions with before/after diagrams.*

Why a linked list?

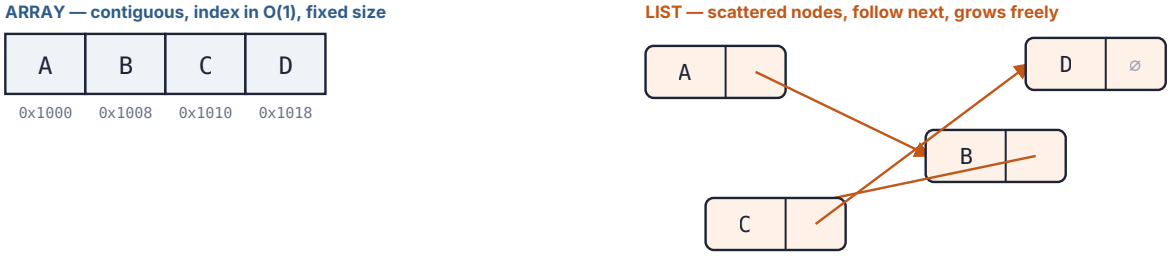


Figure — An array versus a linked list

	ARRAY	LINKED LIST
memory	one contiguous block	one malloc per node, anywhere in the heap
access element i	O(1): `a[i]`	O(i): follow i `next` pointers
insert at front	O(n): shift everything	O(1): one pointer change
insert at end	O(1) if space, else realloc	O(n) to find the last node (O(1) if you keep a tail pointer)
size	fixed or realloc	grows one node at a time

The struct in action

BUILDING A LIST BY HAND

```
1  t_list  *a = ft_lstnew(ft_strdup("one"));    /* node 1 */
2  t_list  *b = ft_lstnew(ft_strdup("two"));    /* node 2 */
3  a->next = b;                                 /* link them   */
4  printf("%s\n", (char *)a->next->content);    /* "two"       */
5  /*        a  ->  next  ->  content
6            |       |         |
7         t_list*  t_list*    void*  → cast to char* to use it  */
```



Figure — a → node("one") → node("two") → NULL

Notice the cast: `content` is a `void *`; to use it as a string you must cast it to `char *`. The list only stores the address — **two separate allocations** exist per node here: the node itself (16 bytes, ft_lstnew) and the string (ft_strdup). That is why freeing a node needs `del` for the content *and* `free` for the node.

t_list * versus t_list **

C passes arguments **by value**: a function receives a *copy*. If a function receives `t_list *head` and does `head = new;`, only its local copy changes; the caller's `head` still points to the old first node. To change the caller's pointer, you pass its address — a pointer to the pointer, `t_list **` — and write through it with `*lst = …`.

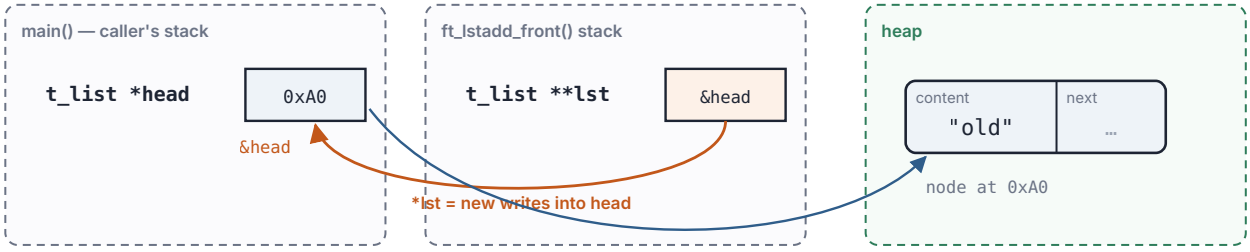


Figure — lst holds the address of head, so *lst = new modifies head itself

FUNCTION	PARAMETER	WHY
lstadd_front	`t_list **lst`	the head changes (new first node)
lstadd_back	`t_list **lst`	the head changes if the list was empty
lstclear	`t_list **lst`	sets the caller's head to NULL afterwards
lstnew, lstsize, lstlast, lstiter, lstdelone, lstmap	`t_list *lst`	they only read the list, or return a new one

KEY IDEA

Three levels to keep straight with `t_list **lst`: `lst` = address of the caller's head variable; `*lst` = the head itself (address of the first node, maybe NULL); `(*lst)->next` = the second node. The parentheses are required: `->` binds tighter than `*`.

ft_lstnew

ft_lstnew.c

LINKED LIST · ALLOCATES (MALLOC)

```
t_list        *ft_lstnew(void *content);
```

Allocates one node, stores `content` (the pointer, not a copy of the data) and sets `next` to NULL. `sizeof(t_list)` = 16 bytes.

FT_LSTNEW.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"

t_list  *ft_lstnew(void *content)
{
    t_list  *node;

    node = malloc(sizeof(t_list));
    if (!node)
        return (NULL);
    node->content = content;
    node->next = NULL;
    return (node);
}
```

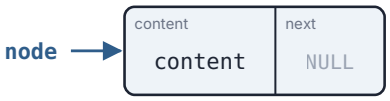


Figure — The fresh node returned by ft_lstnew

PITFALL

`malloc(sizeof(t_list *))` would allocate 8 bytes (a pointer) instead of 16 — a classic heap overflow. `malloc(sizeof(*node))` is a safe alternative that always matches.

ft_lstadd_front

ft_lstadd_front.c

LINKED LIST · NO ALLOCATION

```
void        ft_lstadd_front(t_list **lst, t_list *new);
```

FT_LSTADD_FRONT.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"

void    ft_lstadd_front(t_list **lst, t_list *new)
```

```
{
    if (!lst || !new)
        return ;
    new->next = *lst;
    *lst = new;
}
```

Two pointer assignments, **in this order**. First hook the new node to the current first node, then move the head to the new node. Reversed, you would lose the list (head overwritten before being saved). Works for an empty list too: `new->next = NULL`, head = new. O(1).

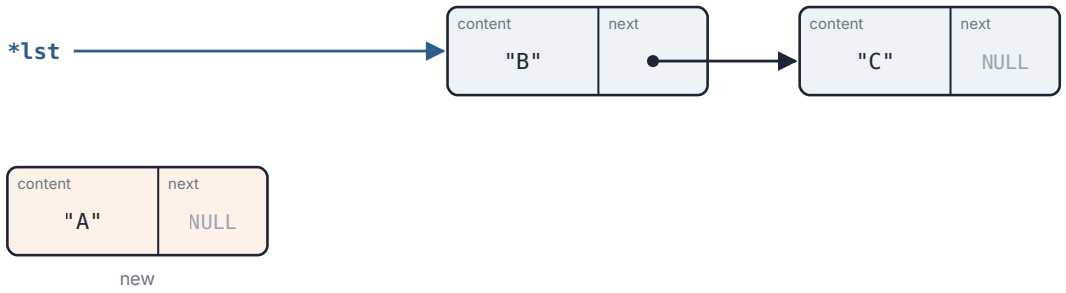


Figure — Before: new is alone



Figure — After: *lst points to new, new→next to the old first node

ft_lstsize

ft_lstsize.c

LINKED LIST NO ALLOCATION

```
unsigned int        ft_lstsize(t_list *lst);
```

FT_LSTSIZE.C · AS WRITTEN IN THE REPOSITORY

```
unsigned int    ft_lstsize(t_list *lst)
{
    t_list          *current;
    unsigned int    size;

    size = 0;
    current = lst;
    while (current)
    {
        current = current->next;
        size++;
    }
    return (size);
```

```
14 }
```

The canonical list traversal: a cursor starts at the head and follows `next` until NULL, counting each step. Empty list (NULL) → 0. The subject prototype returns `int`; this repo returns `unsigned int` (a deviation the evaluator may flag). The file also lacks `#include "libft.h"`, so `t_list` is an unknown type here.

ft_lstlast

ft_lstlast.c

LINKED LIST NO ALLOCATION

```
t_list        *ft_lstlast(t_list *lst);
```

FT_LSTLAST.C · AS WRITTEN IN THE REPOSITORY

```
1  #include "libft.h"
2
3  t_list  *ft_lstlast(t_list *lst)
4  {
5      t_list  *current;
6
7      if (!lst)
8          return (NULL);
9      current = lst;
10     while (current->next)
11         current = current->next;
12     return (current);
13 }
```

Stops **on** the last node, the one whose `next` is NULL — notice the condition is `current->next`, not `current` (that one would walk off the end and return NULL). The initial NULL check is required because `current->next` would dereference NULL.

TRACE — FT_LSTLAST ON A → B → C

ITERATION	CURRENT	CURRENT→NEXT	CONTINUE?
start	A	B	yes
1	B	C	yes
2	C	NULL	no → return C

ft_lstadd_back

ft_lstadd_back.c · uses `ft_lstlast`

LINKED LIST NO ALLOCATION

```
void        ft_lstadd_back(t_list **lst, t_list *new);
```

FT_LSTADD_BACK.C · AS WRITTEN IN THE REPOSITORY

```
1  #include "libft.h"
2
```

```c
void    ft_lstadd_back(t_list **lst, t_list *new)
{
    t_list  *last;

    if (!lst || !new)
        return ;
    if (!(*lst))
    {
        *lst = new;
        return ;
    }
    last = ft_lstlast(*lst);
    last->next = new;
}
```

Two cases. **Empty list**: the new node becomes the head — that is why we need `t_list **`. **Non-empty**: find the last node and link it to new. The head does not change. O(n) because of ft_lstlast.



Figure — After ft_lstadd_back(&head, node("C")): B→next now points to C

ft_lstdelone

ft_lstdelone.c

LINKED LIST NO ALLOCATION

```c
void        ft_lstdelone(t_list *lst, void (*del)(void *));
```

FT_LSTDELONE.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"

void    ft_lstdelone(t_list *lst, void (*del)(void *))
{
    if (!lst)
        return ;
    if (del)
        del(lst->content);
    free(lst);
}
```

Frees **one** node: first its content via the user-supplied `del` (only the user knows how the content was allocated — maybe it is a struct containing more pointers), then the node itself with `free`. It does **not** free `next` and does not relink the list: the caller must save `lst->next` before calling it if the rest of the list is still needed.

PITFALL

Order matters: `free(lst)` then `del(lst->content)` would read freed memory (use-after-free). This repo also tolerates `del == NULL` (frees only the node), a choice for content that was not allocated, e.g. string literals.

ft_lstclear

ft_lstclear.c · uses `ft_lstdelone`

LINKED LIST NO ALLOCATION

```
void        ft_lstclear(t_list **lst, void (*del)(void *));
```

FT_LSTCLEAR.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"

void    ft_lstclear(t_list **lst, void (*del)(void *))
{
    t_list  *current;
    t_list  *next;

    if (!lst)
        return ;
    current = *lst;
    while (current)
    {
        next = current->next;
        ft_lstdelone(current, del);
        current = next;
    }
    *lst = NULL;
}
```

Deletes every node. The crucial line is `next = current->next;` **before** deleting `current` — after ft_lstdelone the node is freed and reading `current->next` would be a use-after-free. At the end, `*lst = NULL` so the caller is not left with a *dangling pointer*.

TRACE — FT_LSTCLEAR ON A → B → C

STEP	CURRENT	NEXT (SAVED)	FREED
1	A	B	A
2	B	C	A B
3	C	NULL	A B C
end	NULL		*lst = NULL

ft_lstiter

ft_lstiter.c

LINKED LIST NO ALLOCATION

```
void        ft_lstiter(t_list *lst, void (*f)(void *));
```

FT_LSTITER.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"

void    ft_lstiter(t_list *lst, void (*f)(void *))
{
    t_list  *current;

    if (!lst || !f)
        return ;
    current = lst;
    while (current)
    {
        f(current->content);
        current = current->next;
    }
}
```

Calls `f` on the **content** of each node (not on the node). The list structure is unchanged; `f` may modify the data the content points to. Example: `ft_lstiter(lst, print_str)`.

ft_lstmap

ft_lstmap.c · uses `ft_lstnew`, `ft_lstadd_back`, `ft_lstclear`

LINKED LIST ALLOCATES (MALLOC)

```
t_list      *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void
*));
```

PARAMETER / RETURN	TYPE	MEANING
`lst`	`t_list *`	source list (not modified)
`f`	`void *(*)(void *)`	makes new content from old content
`del`	`void (*)(void *)`	used to free contents if something fails
`return`	`t_list *`	a brand-new list, or NULL

Builds a new list where each node's content is `f(old_content)`. The original list stays intact. If any allocation fails, everything built so far must be destroyed — including the content just produced by `f` that has no node yet.

FT_LSTMAP.C · AS WRITTEN IN THE REPOSITORY

```c
#include "libft.h"

t_list  *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void *))
{
    t_list  *newlst;
    t_list  *new;
    void    *content;

    if (!lst || !f || !del)
        return (NULL);
    newlst = NULL;
    while (lst)
    {
        content = f(lst->content);
        new = ft_lstnew(content);
        if (!new)
        {
            del(content);
            ft_lstclear(&newlst, del);
            return (NULL);
        }
        if (!newlst)
            newlst = new;
        else
            ft_lstadd_back(&newlst, new);
        lst = lst->next;
    }
    return (newlst);
}
```

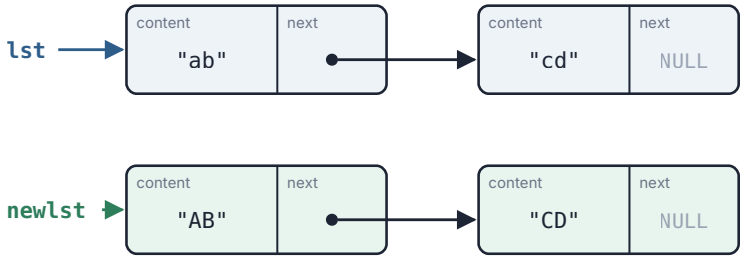


Figure — ft_lstmap(lst, strdup_upper, free) — a new list with new contents

FAILURE PATH, STEP BY STEP

1. `content = f(...)` succeeded — content is a new allocation;
2. `ft_lstnew(content)` fails → returns NULL;
3. `del(content)` — free the orphan content (no node owns it);
4. `ft_lstclear(&newlst, del)` — free all nodes and contents built so far;
5. return NULL. Zero leaks.

> **TIP**
>
> Performance note for the defense: `ft_lstadd_back` walks to the end each time, so building n nodes costs 1 + 2 + … + n = O(n²). Keeping a `last` pointer and doing `last->next = new; last = new;` makes it O(n). The `if (!newlst) newlst = new;` special case is also redundant: ft_lstadd_back already handles an empty list.

PITFALL

If `f` itself returns NULL (its own malloc failed), this version wraps NULL in a node and keeps going. That is what most implementations do; mention it if asked.

QUESTIONS YOU MAY BE ASKED

Q Why does ft_lstadd_front take t_list ** ?
→ Because it must change the caller's head pointer; with t_list * only a local copy would change.

Q What is the difference between lstdelone and lstclear?
→ lstdelone frees one node (content via del, then the node); lstclear frees all nodes from *lst onward and sets *lst to NULL.

Q Why save next before deleting in lstclear?
→ After free, the node's memory is invalid; reading current->next would be a use-after-free.

Q Why does the list need del instead of calling free on content?
→ The list does not know what content is: it could be a literal (must not be freed), a struct with internal allocations, etc. The user provides the right destructor.

Q What does ft_lstlast(NULL) return?
→ NULL.

Q What is the complexity of ft_lstsize, ft_lstadd_front, ft_lstadd_back?
→ O(n), O(1), O(n).

Q What happens to the original list in ft_lstmap?
→ Nothing: it is only read. The result is a completely separate list with new nodes and new contents.

Q How do you print all strings of a list?
→ ft_lstiter(lst, print) where void print(void *c) { ft_putendl_fd((char *)c, 1); }

Q Draw what happens in memory for ft_lstadd_back on an empty list.
→ *lst is NULL, so *lst = new: the caller's head now points to new; new->next is already NULL from lstnew.

CHAPTER 10

The Makefile

> *The Makefile is evaluated before any code is read: if make fails, relinks, or uses the wrong flags, the evaluation stops. The version in this repository has no rules at all; here is what it needs and why.*

MAKEFILE · AS WRITTEN IN THE REPOSITORY

```
NAME = libft.a
CC = cc
CFLAGS = -Werror -Wextra -Werror
AR = ar rcs

SRC = ft_toupper.c \
      ft_tolower.c \
      ft_substr.c  \
      ft_strtrim.c \
      ft_strrchr.c \
      ft_strnstr.c \
      ft_strncmp.c \
      ft_strmapi.c \
      ft_strlen.c  \
      ft_strlcpy.c \
      ft_strlcat.c \
      ft_strjoin.c \
      ft_striteri.c \
      ft_strdup.c \
      ft_strchr.c \
      ft_split.c \
      ft_putstr_fd.c \
      ft_putnbr_fd.c \
      ft_putendl_fd.c \
      ft_putchar_fd.c \
      ft_memset.c \
      ft_memmove.c \
      ft_memcpy.c \
      ft_memcmp.c \
      ft_memchr.c \
      ft_lstsize.c \
      ft_lstnew.c \
      ft_lstmap.c \
      ft_lstlast.c \
      ft_lstiter.c \
      ft_lstdelone.c \
      ft_lstclear.c \
      ft_lstadd_front.c \
      ft_lstadd_back.c \
      ft_itoa.c \
      ft_isprint.c \
      ft_isdigit.c \
      ft_isascii.c \
      ft_isalpha.c \
      ft_isalnum.c \
      ft_calloc.c \
      ft_bzero.c \
      ft_atoi.c
```

IN THIS REPO

- **No rules**: only variables are defined. Running `make` prints "No targets" — the subject requires `$(NAME)`, `all`, `clean`, `fclean`, `re`.
- `CFLAGS = -Werror -Wextra -Werror`: `-Wall` is missing and `-Werror` appears twice.
- No `OBJ` variable, no `.PHONY`.

CORRECTED MAKEFILE (FULL VERSION IN APPENDIX A)

```
NAME    = libft.a
CC      = cc
CFLAGS  = -Wall -Wextra -Werror
AR      = ar rcs
RM      = rm -f

SRC = ft_toupper.c ft_tolower.c … (all 43 files, unchanged)
OBJ = $(SRC:.c=.o)

all: $(NAME)

$(NAME): $(OBJ)
    $(AR) $(NAME) $(OBJ)

%.o: %.c libft.h
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    $(RM) $(OBJ)

fclean: clean
    $(RM) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

How make decides what to do

A rule reads `target: prerequisites` followed by tab-indented commands. make rebuilds a target only if it does not exist or **one of its prerequisites is newer** (file modification times). That is how make avoids recompiling everything — and why a correct Makefile does not **relink**.

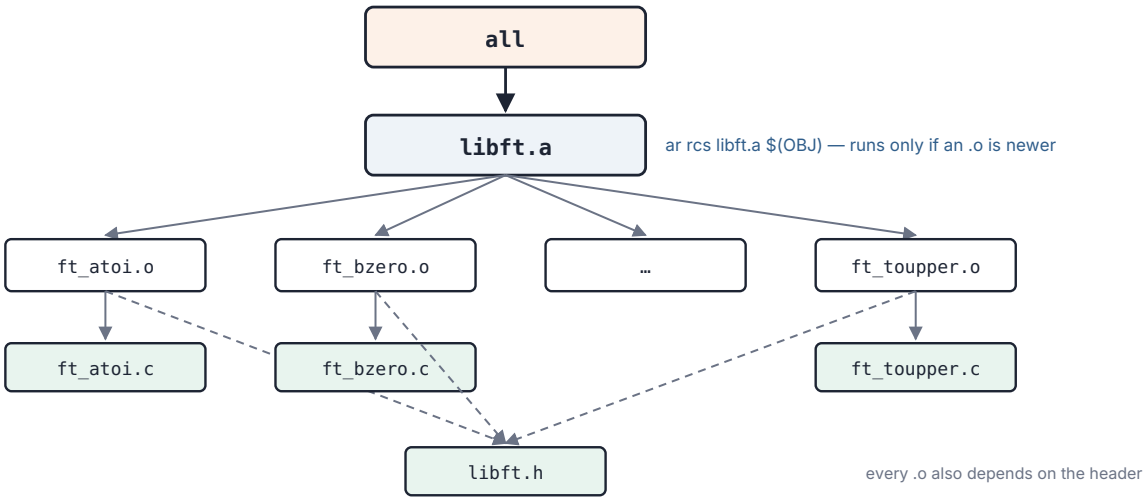


Figure — The dependency tree make walks for `make all`

ELEMENT	MEANING
`OBJ = $(SRC:.c=.o)`	substitution reference: the same list with .c replaced by .o
`%.o: %.c libft.h`	pattern rule: how to build any .o from the .c of the same name; listing libft.h forces a rebuild when the header changes
`$<`	the first prerequisite (the .c file)
`$@`	the target (the .o file)
`ar rcs`	**r**: insert/replace members · **c**: create the archive silently if missing · **s**: write an index of symbols (what ranlib does) so the linker can find functions quickly
`.PHONY`	these targets are not files: run them even if a file called “clean” exists
`re: fclean all`	rebuild from scratch

EVALUATOR ASKS

The relink test: the evaluator runs `make` twice. The second run must print “Nothing to be done for 'all'”. If `$(NAME)` were declared `.PHONY`, or the `ar` command were inside `all` instead of the `$(NAME)` rule, the archive would be rebuilt every time. The corrected Makefile above was tested: second `make` → nothing to be done.

Using the library

COMPILING A PROGRAM AGAINST LIBFT

```
make                    # builds libft.a
cc -Wall -Wextra -Werror main.c -L. -lft -o test
#   -L.   : look for libraries in the current directory
#   -lft  : link libft.a  (the linker adds 'lib' and '.a')
./test
nm libft.a | grep ' T '   # list the functions defined in the archive
```

QUESTIONS YOU MAY BE ASKED

Q What is the difference between clean and fclean?
→ clean removes the object files; fclean also removes libft.a.

Q What is relinking and why is it bad?
→ Rebuilding the target when nothing changed. It wastes time and shows the rules' dependencies are wrong.

Q What does the s in ar rcs do?
→ Writes a symbol index into the archive (like ranlib) so the linker can locate symbols.

Q Why are flags -Wall -Wextra -Werror required?
→ -Wall and -Wextra enable most warnings; -Werror turns them into errors so no warning can be ignored.

Q What is a static library vs a dynamic one?
→ Static (.a): object code copied into the executable at link time. Dynamic (.so/.dylib): loaded at run time and shared by programs.

Q Why use cc and not gcc?
→ The subject asks for cc, the system's default C compiler (gcc or clang depending on the machine).

CHAPTER 11

Audit: What Breaks in This Version

> *Every file was compiled with cc -Wall -Wextra -Werror, checked with norminette, compared against glibc, and run under valgrind with simulated malloc failures. This chapter lists what was found, how serious it is, and the exact fix — the list an evaluator would build.*

SEVERITY	FILE	PROBLEM	FIX
BLOCKING	Makefile	No rules (only variables). `make` → "No targets".	Add OBJ, all, $(NAME), %.o, clean, fclean, re, .PHONY (Chapter 10).
BLOCKING	Makefile	`CFLAGS = -Werror -Wextra -Werror`	`-Wall -Wextra -Werror`
BLOCKING	22 .c files	No `#include "libft.h"`: `size_t`, `NULL`, `t_list`, ft_ functions unknown → compile errors.	Add `#include "libft.h"` to every file (atoi, bzero, calloc, isalpha, isascii, isdigit, isprint, itoa, lstsize, memchr, memcmp, memcpy, memmove, memset, split, strchr, strlcat, strncmp, strnstr, strrchr, tolower, toupper).
BLOCKING	`ft_putstr_fd.c`	Includes `<fcntl.h>` instead of `<unistd.h>` → implicit declaration of write.	Include `<unistd.h>` (or put it in libft.h).
BLOCKING	`ft_strdup.c`	Contains `main()` + `printf`. libft.a would define main: link errors for users; forbidden function.	Delete main and `<stdio.h>`.
BLOCKING	`libft.h`	No prototype for `ft_strjoin`.	Add `char *ft_strjoin(char const *s1, char const *s2);`
WRONG RESULT	`ft_strncmp.c`	Returns non-zero after comparing n equal chars: `ft_strncmp("ab","abc",2)` = −99, libc 0.	`if (i == n) return (0);` before the subtraction.
WRONG RESULT	ft_strchr.c / ft_strrchr.c	Signed `char` compared with `(unsigned char)c`: bytes ≥ 128 never found.	Compare with `(char)c`.
WRONG RESULT	libft.h / ft_lstsize.c	Returns `unsigned int`; subject prototype is `int`.	`int ft_lstsize(t_list *lst)`.
STYLE / MINOR	`libft.h`	`ft_memcmp` declared twice; no blank line after includes (norminette: NL_AFTER_PREPROC).	Remove the duplicate; add an empty line.
STYLE / MINOR	ft_atoi.c, ft_bzero.c, ft_putendl_fd.c	Unused includes (`limits.h`, `ctype.h`, `string.h`, `fcntl.h`).	Remove them.
STYLE / MINOR	`ft_lstmap.c`	O(n²) because of lstadd_back in the loop; redundant `if (!newlst)`.	Keep a last pointer (optional).
STYLE / MINOR	`ft_split.c`	`w_len` computed twice per word.	Store it in a variable (optional).

SEVERITY	FILE	PROBLEM	FIX
STYLE / MINOR	several	Commented-out test mains left in putchar/putstr/putendl/putnbr.	Remove before evaluation.

Compiler output on the original files

CC -WALL -WEXTRA -WERROR -C ON EACH FILE (ABRIDGED)

```
ft_atoi.c:51:24:  error: implicit declaration of function 'ft_isdigit'
ft_bzero.c:17:9:  error: implicit declaration of function 'ft_memset'
ft_calloc.c:14:20: error: unknown type name 'size_t'
ft_lstsize.c:13:28: error: unknown type name 't_list'
ft_memchr.c:13:42: error: unknown type name 'size_t'
ft_putstr_fd.c:20:9: error: implicit declaration of function 'write'
ft_strchr.c:26:17: error: 'NULL' undeclared (first use in this function)
ft_strrchr.c:18:15: error: 'NULL' undeclared (first use in this function)
... (same 'size_t' error in memcmp, memcpy, memmove, memset, split, strlcat, strncmp, strnstr)
```

The three logic fixes, side by side

FT_STRNCMP — ORIGINAL

```
1  while (i < n && *p1 && *p2
2         && (*p1 == *p2))
3  {
4      p1++;
5      p2++;
6      i++;
7  }
8  return (*p1 - *p2);
```

FT_STRNCMP — FIXED

```
1  while (i < n && *p1 && *p2
2         && (*p1 == *p2))
3  {
4      p1++;
5      p2++;
6      i++;
7  }
8  if (i == n)
9      return (0);
10 return (*p1 - *p2);
```

FT_STRCHR — ORIGINAL

```
1  if (*ptr == (unsigned char)c)
2      return (ptr);
3  ...
4  if ((unsigned char)c == '\0')
5      return (ptr);
```

FT_STRCHR — FIXED (SAME IN STRRCHR)

```
1  if (*ptr == (char)c)
2      return (ptr);
3  ...
4  if ((char)c == '\0')
5      return (ptr);
```

Verification of the corrected version

CHECK	RESULT
`make` with corrected Makefile	**builds libft.a**
`make` a second time	**Nothing to be done — no relink**
`norminette *.c libft.h`	**all OK** (original: libft.h fails)
strlcpy / strlcat vs glibc, sizes 0–9	**identical return values and buffers**

CHECK	RESULT
strnstr vs BSD reference, 5 needles × 4 lengths	**identical**
atoi vs glibc incl. INT_MIN, INT_MAX, $\pm 10^{20}$, "--1"	**identical**
strchr / strncmp after fix	**match libc** (before fix: mismatch)
split, strtrim, substr, itoa, strjoin edge cases	**expected results**
ft_split with malloc failing at each of 4 calls (valgrind)	**NULL returned, 0 leaks**
ft_lstmap with malloc failing at each call (valgrind)	**0 leaks**
list functions: build, map, clear (valgrind)	**0 leaks, heads set to NULL**

TIP

How malloc failure was simulated — a technique worth knowing for your own testing: link with `-Wl,--wrap=malloc` and define `__wrap_malloc`, which returns NULL on the k-th call and otherwise forwards to `__real_malloc`. Then loop k = 0, 1, 2… and run under valgrind.

```c
void    *__real_malloc(size_t n);
static int g_fail_at = -1;

void    *__wrap_malloc(size_t n)
{
    if (g_fail_at == 0)
        return (g_fail_at = -1, NULL);
    if (g_fail_at > 0)
        g_fail_at--;
    return (__real_malloc(n));
}
```

CHAPTER 12

Defense Preparation & Recoding Drills

> *How a Libft evaluation goes, the general questions that come up regardless of the function, and a recall card for every function so you can rewrite any of them from memory in front of the evaluator.*

How the evaluation usually goes

1. **Clone** your repository in an empty folder; check that only the required files are there.
2. **Norm**: `norminette` — any error ends the evaluation.
3. **Makefile**: `make`, `make` again (relink?), `make clean`, `make fclean`, `make re`; flags; no wildcards.
4. **Code reading**: the evaluator picks functions and asks you to explain them, line by line.
5. **Tests**: they write a main or use testers (francinette, libftTester, Tripouille) and compare against libc.
6. **Live modification**: sometimes you are asked to rewrite or change a function on the spot.

General questions

QUESTIONS YOU MAY BE ASKED

Q Why recode the libc instead of using it?
→ To understand memory, pointers and edge cases; and because later projects forbid libc functions, so libft becomes your toolbox.

Q What is undefined behaviour? Give examples from libft.
→ Behaviour the standard does not define: anything may happen. Examples: memcpy with overlapping regions, signed int overflow (-INT_MIN), dereferencing NULL, reading past a buffer.

Q Which functions in your library allocate memory?
→ calloc, strdup, substr, strjoin, strtrim, split, itoa, strmapi, lstnew, lstmap. Their results must be freed by the caller.

Q What is a memory leak? How do you check?
→ Heap memory that is no longer referenced but never freed. valgrind --leak-check=full on Linux, leaks on macOS.

Q Why do you test if (!ptr) after malloc?
→ malloc returns NULL when it cannot allocate; writing to NULL crashes.

Q What is a segfault?
→ The OS stops the program because it accessed memory it is not allowed to (NULL, freed, read-only or unmapped).

Q What is the difference between char *s = "abc" and char s[] = "abc"?
→ The first points to a read-only literal; the second is a writable 4-byte array copied onto the stack.

Q What does static mean before a function?
→ Internal linkage: the function is visible only inside its .c file.

Q What does const protect?
→ The pointed-to data from being modified through that pointer (compile-time check).

Q Why size_t for lengths?
→ Unsigned and wide enough for the size of any object; it is what sizeof and strlen produce and malloc takes.

Q What is the difference between ++i and i++?
→ ++i increments then yields the new value; i++ yields the old value then increments. As a standalone statement they are identical.

Q Why does strlcpy exist when strncpy exists?
→ strncpy does not guarantee termination and pads with zeros; strlcpy always terminates and returns the length needed, making truncation easy to detect.

Q How is a negative number stored?
→ Two's complement: invert the bits of the positive value and add 1. That is why int has one more negative value.

Q Can you explain void *?
→ A generic pointer with no element type. Any object pointer converts to and from it implicitly; you must convert it before dereferencing.

What testers check (be ready)

FUNCTION	TRICKY TEST CASES
strlcpy / strlcat	size 0; size smaller than dest length; src empty; exact fit
strchr / strrchr	c = '\0'; c = 'a' + 256 (only low byte counts); character not present
strncmp / memcmp	n = 0; bytes ≥ 128 (\200 vs \0); strings equal for n chars but different after
strnstr	empty needle; len shorter than the match; needle longer than haystack
atoi	"\t\v\f\r\n \f-06050"; "+-1"; INT_MIN; very long digit strings
calloc	count or size 0; SIZE_MAX overflow; result can be freed
memmove	overlap both directions; n = 0
substr	start ≥ strlen; len huge; start + len > strlen
split	empty string; only delimiters; delimiter at both ends; c = '\0'; malloc failure
strtrim	everything trimmed; empty set; set chars in the middle
itoa / putnbr_fd	0, INT_MIN, INT_MAX, negative numbers
lst*	empty list (NULL head); single node; del called exactly once per content; no leaks after lstclear

Recall cards — rewrite from memory

Each card is the minimal skeleton of the logic. Cover the right column, try to write the function, then compare.

FUNCTION	SKELETON
`ft_strlen`	`i = 0; while (s[i]) i++; return i;`

FUNCTION	SKELETON
ft_memset	p = s; while (i < n) p[i++] = (unsigned char)c; return s;
ft_bzero	ft_memset(s, 0, n);
ft_memcpy	d = dest; s = src; while (i < n) { d[i] = s[i]; i++; } return dest;
ft_memmove	if (d == s || !n) return; if (d > s) while (n--) d[n] = s[n]; else while (n--) *d++ = *s++;
ft_memchr	while (i < n) { if (p[i] == (unsigned char)c) return &p[i]; i++; } return NULL;
ft_memcmp	while (i < n) { if (p1[i] != p2[i]) return p1[i] - p2[i]; i++; } return 0;
ft_calloc	if (size && count > SIZE_MAX / size) return NULL; p = malloc(count*size); bzero; return p;
ft_strlcpy	len = strlen(src); if (!size) return len; copy while (src[i] && i < size-1); dest[i]=0; return len;
ft_strlcat	dlen = len of dest bounded by size; if (dlen == size) return size + slen; copy while i < size-dlen-1; terminate; return dlen + slen;
ft_strchr	while (*s) { if (*s == (char)c) return s; s++; } return ((char)c == 0) ? s : NULL;
ft_strrchr	last = NULL; walk: if match, last = &s[i]; at end: if c == 0 return &s[i]; return last;
ft_strncmp	while (i < n && s1[i] && s1[i] == s2[i]) i++; if (i == n) return 0; return (uc)s1[i] - (uc)s2[i];
ft_strnstr	if (!*needle) return hay; for each start j < len: compare while j+i < len; if needle ended → return hay + j;
ft_strdup	p = malloc(len + 1); copy; terminate; return p;
ft_atoi	skip 9–13 and ' '; one sign; while digit: r = r*10 + (c - '0'); return r * sign;
ft_substr	if start ≥ slen → count 0; else count = min(len, slen - start); malloc(count+1); copy; terminate;
ft_strjoin	malloc(l1 + l2 + 1); copy s1; copy s2; terminate;
ft_strtrim	start: skip set chars; end: from last char back while in set and end > start; substr(start, end-start+1);
ft_split	count words (state flag); malloc(count+1 pointers); for each word: substr; on failure free all; arr[i] = NULL;
ft_itoa	long nb = n; len = digits (+1 if neg); malloc(len+1); fill from the end with nb % 10; '-' at 0;
ft_strmapi	r = malloc(len+1); r[i] = f(i, s[i]); terminate;
ft_striteri	while (s[i]) { f(i, &s[i]); i++; }
ft_putnbr_fd	if INT_MIN print literal; if n < 0 { '-'; n = -n; } if n ≥ 10 recurse(n/10); putchar(n%10 + '0');
ft_lstnew	n = malloc(sizeof(t_list)); n->content = content; n->next = NULL;
ft_lstadd_front	new->next = *lst; *lst = new;
ft_lstadd_back	if (!*lst) *lst = new; else ft_lstlast(*lst)->next = new;
ft_lstsize / lstlast	walk with a cursor; count / stop when cursor->next == NULL
ft_lstdelone	del(lst->content); free(lst);
ft_lstclear	while (cur) { next = cur->next; delone(cur, del); cur = next; } *lst = NULL;
ft_lstiter	while (lst) { f(lst->content); lst = lst->next; }

FUNCTION	SKELETON
`ft_lstmap`	for each node: c = f(content); n = lstnew(c); if (!n) { del(c); lstclear(&new, del); return NULL; } add_back;

APPENDIX A

Complete Corrected Source

> *Every file of the library after the fixes from Chapter 11. It builds with cc -Wall -Wextra -Werror, passes norminette, and matches libc on the tests described. Highlighted lines differ from the repository. The 42 header comment is omitted to save space — keep yours in each file.*

LIBFT.H

```
#ifndef LIBFT_H
# define LIBFT_H
# include <stddef.h>
# include <stdlib.h>
# include <unistd.h>
# include <stdint.h>

typedef struct s_list
{
    void            *content;
    struct s_list   *next;
}                   t_list;

int                 ft_atoi(const char *str);
int                 ft_toupper(int c);
int                 ft_tolower(int c);
int                 ft_strncmp(const char *s1, const char *s2, size_t n);
int                 ft_memcmp(const void *s1, const void *s2, size_t n);
int                 ft_isprint(int c);
int                 ft_isdigit(int c);
int                 ft_isascii(int c);
int                 ft_isalpha(int c);
int                 ft_isalnum(int c);
int                 ft_lstsize(t_list *lst);
size_t              ft_strlen(char const *s);
size_t              ft_strlcpy(char *dest, const char *src, size_t size);
size_t              ft_strlcat(char *dest, const char *src, size_t size);
char                *ft_substr(char const *s, unsigned int start, size_t len);
char                *ft_strtrim(char const *s1, char const *set);
char                *ft_strchr(const char *s, int c);
char                *ft_strrchr(char const *s, int c);
char                *ft_strnstr(const char *haystack, const char *needle,
                        size_t len);
char                *ft_strmapi(char const *s, char (*f)(unsigned int, char));
char                *ft_strjoin(char const *s1, char const *s2);
char                *ft_strdup(const char *s);
char                *ft_itoa(int n);
char                **ft_split(char const *s, char c);
void                ft_striteri(char *s, void (*f)(unsigned int, char *));
void                ft_putchar_fd(char c, int fd);
void                ft_putstr_fd(char *s, int fd);
void                ft_putnbr_fd(int n, int fd);
void                ft_putendl_fd(char *s, int fd);
void                ft_bzero(void *s, size_t n);
void                *ft_memset(void *s, int c, size_t len);
void                *ft_memchr(const void *s, int c, size_t n);
void                *ft_memmove(void *dest, const void *src, size_t n);
void                *ft_memcpy(void *dest, const void *src, size_t n);
void                *ft_calloc(size_t count, size_t size);
void                ft_lstiter(t_list *lst, void (*f)(void *));
void                ft_lstdelone(t_list *lst, void (*del)(void *));
void                ft_lstclear(t_list **lst, void (*del)(void *));
void                ft_lstadd_front(t_list **lst, t_list *new);
void                ft_lstadd_back(t_list **lst, t_list *new);
t_list              *ft_lstnew(void *content);
t_list              *ft_lstmap(t_list *lst, void *(*f)(void *),
                        void (*del)(void *));
```

```
t_list              *ft_lstlast(t_list *lst);
#endif
```

MAKEFILE

```
NAME    = libft.a
CC      = cc
CFLAGS  = -Wall -Wextra -Werror
AR      = ar rcs
RM      = rm -f

SRC = ft_toupper.c \
      ft_tolower.c \
      ft_substr.c  \
      ft_strtrim.c \
      ft_strrchr.c \
      ft_strnstr.c \
      ft_strncmp.c \
      ft_strmapi.c \
      ft_strlen.c  \
      ft_strlcpy.c \
      ft_strlcat.c \
      ft_strjoin.c \
      ft_striteri.c \
      ft_strdup.c \
      ft_strchr.c \
      ft_split.c \
      ft_putstr_fd.c \
      ft_putnbr_fd.c \
      ft_putendl_fd.c \
      ft_putchar_fd.c \
      ft_memset.c \
      ft_memmove.c \
      ft_memcpy.c \
      ft_memcmp.c \
      ft_memchr.c \
      ft_lstsize.c \
      ft_lstnew.c \
      ft_lstmap.c \
      ft_lstlast.c \
      ft_lstiter.c \
      ft_lstdelone.c \
      ft_lstclear.c \
      ft_lstadd_front.c \
      ft_lstadd_back.c \
      ft_itoa.c \
      ft_isprint.c \
      ft_isdigit.c \
      ft_isascii.c \
      ft_isalpha.c \
      ft_isalnum.c \
      ft_calloc.c \
      ft_bzero.c \
      ft_atoi.c

OBJ = $(SRC:.c=.o)

all: $(NAME)

$(NAME): $(OBJ)
    $(AR) $(NAME) $(OBJ)

%.o: %.c libft.h
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    $(RM) $(OBJ)

fclean: clean
    $(RM) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

FT_ISALPHA.C

```c
#include "libft.h"

int ft_isalpha(int c)
{
    if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z'))
        return (1);
    return (0);
}
```

FT_ISDIGIT.C

```c
#include "libft.h"

int ft_isdigit(int c)
{
    if (c >= '0' && c <= '9')
        return (1);
    return (0);
}
```

FT_ISALNUM.C

```c
#include "libft.h"

int ft_isalnum(int c)
{
    return (ft_isalpha(c) || ft_isdigit(c));
}
```

FT_ISASCII.C

```c
#include "libft.h"

int ft_isascii(int c)
{
    if (c >= 0 && c <= 127)
        return (1);
    return (0);
}
```

FT_ISPRINT.C

```c
#include "libft.h"

int ft_isprint(int c)
{
    if (c >= 32 && c <= 126)
        return (1);
    return (0);
}
```

FT_TOUPPER.C

```c
#include "libft.h"

int ft_toupper(int c)
{
    if (c >= 'a' && c <= 'z')
        return (c - 32);
    return (c);
}
```

FT_TOLOWER.C

```c
#include "libft.h"

int ft_tolower(int c)
{
    if (c >= 'A' && c <= 'Z')
        return (c + 32);
    return (c);
}
```

FT_MEMSET.C

```c
#include "libft.h"

void    *ft_memset(void *s, int c, size_t len)
{
    unsigned char   *p;
    size_t          i;

    p = s;
    i = 0;
    while (i < len)
        p[i++] = (unsigned char)c;
    return (s);
}
```

FT_BZERO.C

```c
#include "libft.h"

void    ft_bzero(void *s, size_t n)
{
    ft_memset(s, 0, n);
}
```

FT_MEMCPY.C

```c
#include "libft.h"

void    *ft_memcpy(void *dest, const void *src, size_t n)
{
    unsigned char       *d;
    const unsigned char *s;
    size_t              i;

    d = (unsigned char *)dest;
    s = (const unsigned char *)src;
    i = 0;
    while (i < n)
    {
        d[i] = s[i];
        i++;
    }
    return (dest);
}
```

FT_MEMMOVE.C

```c
#include "libft.h"

void    *ft_memmove(void *dest, const void *src, size_t n)
{
    unsigned char       *d;
    const unsigned char *s;

    d = dest;
    s = src;
    if (d == s || n == 0)
        return (dest);
    if (d > s)
    {
```

```
        while (n--)
            d[n] = s[n];
    }
    else
    {
        while (n--)
            *d++ = *s++;
    }
    return (dest);
}
```

FT_MEMCHR.C

```
#include "libft.h"

void    *ft_memchr(const void *s, int c, size_t n)
{
    const unsigned char *ptr;
    size_t              i;

    ptr = (const unsigned char *)s;
    i = 0;
    while (i < n)
    {
        if (ptr[i] == (unsigned char)c)
            return ((void *)&ptr[i]);
        i++;
    }
    return (NULL);
}
```

FT_MEMCMP.C

```
#include "libft.h"

int ft_memcmp(const void *s1, const void *s2, size_t n)
{
    const unsigned char *p1;
    const unsigned char *p2;
    size_t              i;

    if (n == 0)
        return (0);
    p1 = (const unsigned char *)s1;
    p2 = (const unsigned char *)s2;
    i = 0;
    while (i < n && (*p1 == *p2))
    {
        p1++;
        p2++;
        i++;
    }
    if (i == n)
        return (0);
    return (*p1 - *p2);
}
```

FT_CALLOC.C

```
#include <stdint.h>

#include "libft.h"

void    *ft_calloc(size_t count, size_t size)
{
    size_t          len;
    unsigned char   *ptr;

    if (size != 0 && count > SIZE_MAX / size)
        return (NULL);
    len = count * size;
    if (!len)
```

```
        len = 1;
    ptr = malloc(len);
    if (!ptr)
        return (NULL);
    ft_bzero(ptr, len);
    return ((void *)ptr);
}
```

FT_STRLEN.C

```c
#include "libft.h"

size_t  ft_strlen(char const *s)
{
    size_t  i;

    i = 0;
    while (s[i])
        i++;
    return (i);
}
```

FT_STRLCPY.C

```c
#include "libft.h"

size_t  ft_strlcpy(char *dest, const char *src, size_t size)
{
    size_t  slen;
    size_t  i;

    slen = ft_strlen(src);
    if (size == 0)
        return (slen);
    i = 0;
    while (src[i] && i < size - 1)
    {
        dest[i] = src[i];
        i++;
    }
    dest[i] = '\0';
    return (slen);
}
```

FT_STRLCAT.C

```c
#include "libft.h"

size_t  ft_strlcat(char *dest, const char *src, size_t size)
{
    size_t  dlen;
    size_t  slen;
    size_t  i;
    char    *dp;

    dlen = 0;
    slen = ft_strlen(src);
    while (dlen < size && dest[dlen])
        dlen++;
    if (size == 0)
        return (slen);
    if (dlen == size)
        return (size + slen);
    i = 0;
    dp = dest + dlen;
    while (src[i] && i < size - dlen - 1)
        *dp++ = src[i++];
    *dp = '\0';
    return (dlen + slen);
}
```

FT_STRCHR.C

```c
#include "libft.h"

char    *ft_strchr(const char *s, int c)
{
    char    *ptr;

    ptr = (char *)s;
    while (*ptr)
    {
        if (*ptr == (char)c)
            return (ptr);
        ptr++;
    }
    if ((char)c == '\0')
        return (ptr);
    return (NULL);
}
```

FT_STRRCHR.C

```c
#include "libft.h"

char    *ft_strrchr(char const *s, int c)
{
    const char  *ptr;
    size_t      i;

    ptr = NULL;
    i = 0;
    while (s[i])
    {
        if (s[i] == (char)c)
            ptr = &s[i];
        i++;
    }
    if ((char)c == '\0')
        ptr = &s[i];
    return ((char *)ptr);
}
```

FT_STRNCMP.C

```c
#include "libft.h"

int ft_strncmp(const char *s1, const char *s2, size_t n)
{
    size_t              i;
    const unsigned char *p1;
    const unsigned char *p2;

    if (n == 0)
        return (0);
    i = 0;
    p1 = (const unsigned char *)s1;
    p2 = (const unsigned char *)s2;
    while (i < n && *p1 && *p2 && (*p1 == *p2))
    {
        p1++;
        p2++;
        i++;
    }
    if (i == n)
        return (0);
    return (*p1 - *p2);
}
```

FT_STRNSTR.C

```c
#include "libft.h"
```

```c

char	*ft_strnstr(const char *haystack, const char *needle, size_t len)
{
	const char	*big;
	const char	*little;
	size_t		i;
	size_t		j;

	if (!*needle)
		return ((char *)haystack);
	j = 0;
	while (j < len && *haystack)
	{
		i = 0;
		big = haystack;
		little = needle;
		while (j + i < len && *big && *little && (*big == *little))
		{
			big++;
			little++;
			i++;
		}
		if (*little == '\0')
			return ((char *)haystack);
		j++;
		haystack++;
	}
	return (NULL);
}
```

FT_STRDUP.C

```c
#include "libft.h"

char	*ft_strdup(const char *s)
{
	size_t	slen;
	char	*dup;

	if (!s)
		return (NULL);
	slen = ft_strlen(s);
	dup = malloc(slen + 1);
	if (!dup)
		return (NULL);
	ft_strlcpy(dup, s, slen + 1);
	return (dup);
}
```

FT_ATOI.C

```c
#include "libft.h"

static int	issign(unsigned char c)
{
	if (c == '-' || c == '+')
		return (1);
	return (0);
}

static int	ft_isspace(unsigned char c)
{
	if ((c >= 9 && c <= 13) || c == ' ')
		return (1);
	return (0);
}

static int	ft_overflow(int sign)
{
	if (sign == -1)
		return (0);
	return (-1);
}

```

```c
int ft_atoi(const char *str)
{
    long long   result;
    int         sign;

    result = 0;
    sign = 1;
    while (*str && ft_isspace(*str))
        str++;
    if (issign(*str))
    {
        if (*str == '-')
            sign = -1;
        str++;
    }
    while (*str && ft_isdigit(*str))
    {
        if (result > (9223372036854775807 - (*str - '0')) / 10)
            return (ft_overflow(sign));
        result = result * 10 + (*str - '0');
        str++;
    }
    return ((int)(result * sign));
}
```

FT_SUBSTR.C

```c
#include "libft.h"

static size_t   guard(size_t len, size_t slen, unsigned int start)
{
    size_t  count;

    if (start >= slen)
        count = 0;
    else if (len > slen - start)
        count = slen - start;
    else
        count = len;
    return (count);
}

char    *ft_substr(char const *s, unsigned int start, size_t len)
{
    char    *sub;
    size_t  i;
    size_t  count;
    size_t  slen;

    if (!s)
        return (NULL);
    slen = ft_strlen(s);
    count = guard(len, slen, start);
    sub = malloc(count + 1);
    if (!sub)
        return (NULL);
    i = 0;
    while (i < count)
    {
        sub[i] = s[start + i];
        i++;
    }
    sub[i] = '\0';
    return (sub);
}
```

FT_STRJOIN.C

```c
#include "libft.h"
#include <stdint.h>

char    *ft_strjoin(char const *s1, char const *s2)
{
    size_t  s1len;
```

```c
    size_t  s2len;
    char    *joined;
    char    *ptr;

    if (!s1 || !s2)
        return (NULL);
    s1len = ft_strlen(s1);
    s2len = ft_strlen(s2);
    if (s2len == SIZE_MAX || s1len > SIZE_MAX - s2len - 1)
        return (NULL);
    joined = malloc(s1len + s2len + 1);
    if (!joined)
        return (NULL);
    ptr = joined;
    while (*s1)
        *ptr++ = *s1++;
    while (*s2)
        *ptr++ = *s2++;
    *ptr = '\0';
    return (joined);
}
```

FT_STRTRIM.C

```c
#include "libft.h"

char    *ft_strtrim(char const *s1, char const *set)
{
    size_t  start;
    size_t  end;

    if (!s1 || !set)
        return (NULL);
    if (ft_strlen(s1) == 0)
        return (ft_strdup(s1));
    start = 0;
    while (s1[start] && ft_strchr(set, s1[start]))
        start++;
    if (s1[start] == '\0')
        return (ft_strdup(""));
    end = ft_strlen(s1) - 1;
    while (end > start && ft_strchr(set, s1[end]))
        end--;
    return (ft_substr(s1, start, end - start + 1));
}
```

FT_SPLIT.C

```c
#include "libft.h"

static size_t   w_count(const char *s, char c)
{
    int     found;
    size_t  count;

    count = 0;
    found = 0;
    while (*s)
    {
        if (*s == c)
            found = 0;
        else if (found == 0)
        {
            found = 1;
            count++;
        }
        s++;
    }
    return (count);
}

static void free_arr(char **arr, size_t n)
{
    size_t  i;
```

```
    i = 0;
    while (i < n)
    {
        free(arr[i]);
        i++;
    }
    free(arr);
}

static size_t   w_len(char const *s, char c)
{
    size_t  len;

    len = 0;
    while (*s && *s != c)
    {
        len++;
        s++;
    }
    return (len);
}

char    **ft_split(char const *s, char c)
{
    char    **arr;
    size_t  i;

    if (!s)
        return (NULL);
    arr = malloc(sizeof(char *) * (w_count(s, c) + 1));
    if (!arr)
        return (NULL);
    i = 0;
    while (*s)
    {
        if (*s != c)
        {
            arr[i] = ft_substr(s, 0, w_len(s, c));
            if (!arr[i])
                return (free_arr(arr, i), NULL);
            i++;
            s += w_len(s, c);
        }
        else
            s++;
    }
    arr[i] = NULL;
    return (arr);
}
```

FT_ITOA.C

```
#include "libft.h"

static size_t   ft_intlen(long nbr)
{
    size_t  len;

    len = 0;
    if (nbr == 0)
        return (1);
    if (nbr < 0)
    {
        len++;
        nbr = -nbr;
    }
    while (nbr > 0)
    {
        len++;
        nbr /= 10;
    }
    return (len);
}

char    *ft_itoa(int n)
```

```c
{
    char    *ptr;
    size_t  nlen;
    long    nbr;

    nbr = n;
    nlen = ft_intlen(nbr);
    ptr = malloc(nlen + 1);
    if (!ptr)
        return (NULL);
    ptr[nlen] = '\0';
    if (nbr < 0)
    {
        ptr[0] = '-';
        nbr = -nbr;
    }
    if (nbr == 0)
        ptr[0] = '0';
    while (nbr > 0)
    {
        ptr[--nlen] = nbr % 10 + '0';
        nbr /= 10;
    }
    return (ptr);
}
```

FT_STRMAPI.C

```c
#include "libft.h"

char    *ft_strmapi(char const *s, char (*f)(unsigned int, char))
{
    char    *str;
    size_t  i;
    size_t  slen;

    if (!s || !f)
        return (NULL);
    slen = ft_strlen(s);
    str = malloc(slen + 1);
    if (!str)
        return (NULL);
    i = 0;
    while (i < slen)
    {
        str[i] = f(i, s[i]);
        i++;
    }
    str[i] = '\0';
    return (str);
}
```

FT_STRITERI.C

```c
#include "libft.h"

void    ft_striteri(char *s, void (*f)(unsigned int, char *))
{
    unsigned int    i;

    if (!s || !f)
        return ;
    i = 0;
    while (s[i])
    {
        f(i, &s[i]);
        i++;
    }
}
```

FT_PUTCHAR_FD.C

```c
#include "libft.h"
#include <unistd.h>

void    ft_putchar_fd(char c, int fd)
{
    write(fd, &c, 1);
}
```

FT_PUTSTR_FD.C

```c
#include "libft.h"
#include <unistd.h>

void    ft_putstr_fd(char *s, int fd)
{
    if (!s)
        return ;
    write(fd, s, ft_strlen(s));
}
```

FT_PUTENDL_FD.C

```c
#include "libft.h"

void    ft_putendl_fd(char *s, int fd)
{
    if (!s)
        return ;
    ft_putstr_fd(s, fd);
    ft_putchar_fd('\n', fd);
}
```

FT_PUTNBR_FD.C

```c
#include "libft.h"

void    ft_putnbr_fd(int n, int fd)
{
    char    c;

    if (n == -2147483648)
    {
        ft_putstr_fd("-2147483648", fd);
        return ;
    }
    if (n < 0)
    {
        ft_putchar_fd('-', fd);
        n = -n;
    }
    if (n >= 10)
        ft_putnbr_fd(n / 10, fd);
    c = n % 10 + '0';
    ft_putchar_fd(c, fd);
}
```

FT_LSTNEW.C

```c
#include "libft.h"

t_list  *ft_lstnew(void *content)
{
    t_list  *node;

    node = malloc(sizeof(t_list));
    if (!node)
        return (NULL);
    node->content = content;
    node->next = NULL;
    return (node);
```

```
}
```

FT_LSTADD_FRONT.C

```c
#include "libft.h"

void    ft_lstadd_front(t_list **lst, t_list *new)
{
    if (!lst || !new)
        return ;
    new->next = *lst;
    *lst = new;
}
```

FT_LSTSIZE.C

```c
#include "libft.h"

int ft_lstsize(t_list *lst)
{
    t_list          *current;
    int             size;

    size = 0;
    current = lst;
    while (current)
    {
        current = current->next;
        size++;
    }
    return (size);
}
```

FT_LSTLAST.C

```c
#include "libft.h"

t_list  *ft_lstlast(t_list *lst)
{
    t_list  *current;

    if (!lst)
        return (NULL);
    current = lst;
    while (current->next)
        current = current->next;
    return (current);
}
```

FT_LSTADD_BACK.C

```c
#include "libft.h"

void    ft_lstadd_back(t_list **lst, t_list *new)
{
    t_list  *last;

    if (!lst || !new)
        return ;
    if (!(*lst))
    {
        *lst = new;
        return ;
    }
    last = ft_lstlast(*lst);
    last->next = new;
}
```

FT_LSTDELONE.C

```c
#include "libft.h"

void    ft_lstdelone(t_list *lst, void (*del)(void *))
{
    if (!lst)
        return ;
    if (del)
        del(lst->content);
    free(lst);
}
```

FT_LSTCLEAR.C

```c
#include "libft.h"

void    ft_lstclear(t_list **lst, void (*del)(void *))
{
    t_list  *current;
    t_list  *next;

    if (!lst)
        return ;
    current = *lst;
    while (current)
    {
        next = current->next;
        ft_lstdelone(current, del);
        current = next;
    }
    *lst = NULL;
}
```

FT_LSTITER.C

```c
#include "libft.h"

void    ft_lstiter(t_list *lst, void (*f)(void *))
{
    t_list  *current;

    if (!lst || !f)
        return ;
    current = lst;
    while (current)
    {
        f(current->content);
        current = current->next;
    }
}
```

FT_LSTMAP.C

```c
#include "libft.h"

t_list  *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void *))
{
    t_list  *newlst;
    t_list  *new;
    void    *content;

    if (!lst || !f || !del)
        return (NULL);
    newlst = NULL;
    while (lst)
    {
        content = f(lst->content);
        new = ft_lstnew(content);
        if (!new)
        {
```

```
            del(content);
            ft_lstclear(&newlst, del);
            return (NULL);
        }
        if (!newlst)
            newlst = new;
        else
            ft_lstadd_back(&newlst, new);
        lst = lst->next;
    }
    return (newlst);
}
```